

Re2creds: Reusable Anonymous Credentials from Malleable NIZK and Legacy Signatures

Bin Xie, Tianyu Zheng, Rui Song, Shang Gao, Bin Xiao
Department of Computing, The Hong Kong Polytechnic University

Abstract

Decentralized identity is revolutionizing secure digital interactions by giving users control over their personal data. Anonymous credentials (ACs) are fundamental to this paradigm, yet their practical application is hindered by significant usability and efficiency challenges. Existing AC systems often struggle with limitations in predicate expressiveness, privacy protection, and incompatibility with widely adopted legacy signatures based on recommended curves. To overcome these obstacles, this paper introduces a novel AC system named Re2creds. Re2creds establishes a new paradigm of reusable credential presentation, which drastically cuts computational costs by allowing the core of a presentation to be reused across multiple sessions with only lightweight updates. Furthermore, Re2creds incorporates a proof combination mechanism that efficiently supports legacy signatures by moving the most computationally intensive cryptographic operations outside the arithmetic circuit. This approach makes it practical to use credentials based on NIST-recommended curves, removing a critical barrier to real-world adoption. We demonstrate Re2creds's security properties through a refined UC ideal functionality, accompanied by rigorous proofs. Experimental evaluations demonstrate significant performance improvements over existing schemes: credential generation time decreases by more than 50% when deriving from an existing presentation. Additionally, Re2creds makes the presentation of legacy signatures feasible compared to other ACs, which takes less than 1s for a BLS signature based on BN254.

I. INTRODUCTION

Identity management forms the cornerstone of secure digital interactions while current systems usually resort to centralized authorities for authentication. Rising concerns over data privacy and centralized control have sparked demand for personal data sovereignty. This shift has driven the development of decentralized identity, a system that moves beyond traditional custodial models by leveraging verifiable credentials, cryptography, and blockchain technology.

The core technology of decentralized identity systems relies on anonymous credentials (ACs). This cryptographic primitive involves three roles: the issuer, the holder, and the verifier. The process starts when the issuer issues verifiable credentials to the holder, cryptographically endorsing specific attributes (e.g., birthday and driving license class). A key feature is that holders maintain control over the authentication process by cryptographically justifying specific predicates (such as $\text{age} \geq 18$), while ensuring credential and identity privacy. ACs serve as the foundation for privacy-centric applications in Web3. The security and privacy guarantees afforded by ACs underpin innovations such as anonymous web browsing [Rosenberg et al.(2023)], privacy-preserving blacklisting mechanisms [Rosenberg et al.(2022)], and verifiable assertions to combat misinformation [Naveh and Tromer(2016)].

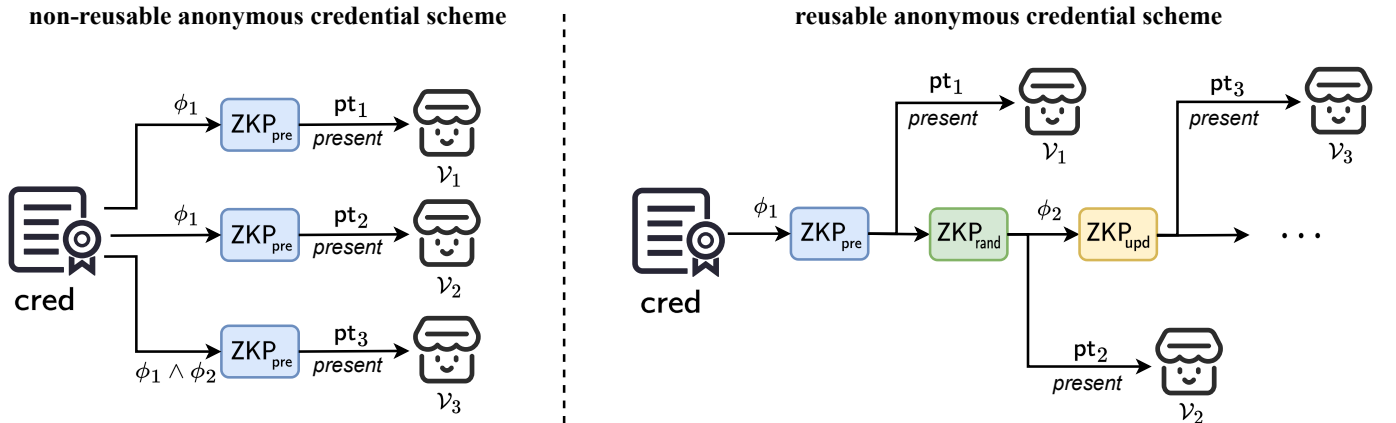


Fig. 1: Different anonymous credential models.

Despite attracting significant interest from industry and academic practitioners, the practical deployment of ACs remains limited. Existing ACs suffer from fundamental issues in usability and efficiency that curtail their real-world adoption. How to construct efficient and practical ACs with strong privacy guarantees remains an open problem.

Previous solutions and limitations. Many classic AC implementations rely on specialized signature schemes, such as BBS+, URS, PS, and SPS [Du et al.(2023)], [Shi et al.(2023)], [Sonnino et al.(2018)], [Xie et al.(2024)]. These schemes are designed

to integrate seamlessly with signature rerandomization mechanisms and bespoke Sigma protocols [Attema and Cramer(2020)], enabling efficient credential presentations. This efficiency, however, comes at the cost of expressiveness and incomplete privacy protection; such systems either lack support for Turing-complete predicates, which can only prove a limited set of predicates over attributes, or require disclosing the concrete attribute values for predicate checks. Furthermore, these schemes often result in large presentation sizes and impose high verification costs, rendering them impractical for resource-constrained verifiers, such as those used in on-chain verification.

Another technical route leverages *general-purpose zk-SNARKs* to construct ACs [Wang et al.(2023)], [Song(2023)], enabling the presentation of credentials built from widely deployed signatures (e.g., ECDSA) in a privacy-preserving way. These schemes benefit from the inherent succinctness of zk-SNARKs and their ability to support arbitrary relations (i.e., predicates). Consequently, these solutions can achieve small presentation sizes and low verification costs, making the presentation’s verification a trivial task. However, the efficiency of generating presentations remains a significant challenge for widely deployed legacy signature schemes based on recommended curves (e.g., P-256 and BN254), as the proof generation process usually necessitates tens of seconds in a high-performance computing environment. While several studies [Woo et al.(2025)] have attempted to mitigate this issue by employing state-of-the-art circuit optimization techniques, the challenge of burdensome presentation generation persists.

TABLE I: Comparisons among different ACs.

Schemes	Adaption to Recommended Curves	Turing-Complete Predicate	Attribute Hiding	Re-randomizable Presentation	Updatable Predicate
UCBlocker [Du et al.(2023)]	✓	✓	✗	✗	✓
TSC’23 [Shi et al.(2023)]	✓	✓	✗	✓	✗
Coconut [Sonnino et al.(2018)]	✓	✗	✓	✗	✓
AccCred [Xie et al.(2024)]	✓	✗	✓	✗	✓
Hades [Wang et al.(2023)]	✗	✓	✓	✓	✗
LinkDID [Song(2023)]	✗	✓	✓	✗	✗
Re2creds	✓	✓	✓	✓	✓

¹ Re-randomizable presentation allows efficient rerandomization of existing ones rather than burdensome regeneration.

² Updatable predicate allows efficiently appending additional predicate assertions to existing presentations without generating a new presentation from scratch.

Motivation. To address these limitations, this paper introduces a novel anonymous credential system called Re2creds, which stands for *Reusable anonymous credential scheme based on Recursive SNARKs*. It is designed for the secure, efficient, and privacy-preserving presentation of credentials, while simultaneously adapting to the widespread legacy signatures and enhancing presentation efficiency.

One of our main ideas is a paradigm shift in how credential presentations are generated. As illustrated in Figure 1 (left), the conventional model requires a credential holder to generate a new presentation token for each authentication session, even if the verifier’s predicates are identical ($\mathcal{V}_1$ and $\mathcal{V}_2$) or have been incrementally modified from a previous session ($\mathcal{V}_2$ and $\mathcal{V}_3$). In contrast, our system employs *reusable credential presentation*. This paradigm allows the majority of the computational work involved in a presentation to be reused across sessions. Consequently, a practical and efficient AC can be constructed by focusing on lightweight rerandomization and predicate updates for credential presentations, as illustrated in Figure 1 (right). To this end, we introduce a generalized malleable NIZK construction called *Malo* (abbreviation of Malleable Halo), which can efficiently randomize existing proofs for presentations of $\mathcal{V}_2$ and allows efficiently aggregating a new predicate into existing proofs of $\mathcal{V}_3$.

The other idea is devoted to solving the efficiency issue of presenting legacy signatures. Existing general zk-SNARK-based ACs require the entire signature verification process to be performed in a circuit, which typically includes a series of expensive operations such as group point exponentiation and pairing. This proof of possession (PoP) mechanism [Nguyen et al.(1999)] is only applicable to signatures built on specific curves, such as Baby Jubjub, because they can be efficiently encoded in a circuit. To enable efficient presentation of signatures based on other curves, we adopt a proof composition mechanism [Campanelli et al.(2019)]. For some arithmetic computations, such as algebraic hashing [Grassi et al.(2021)], native field operations, and predicate checks, we still use circuits for proof generation. For the remaining expensive computations, such as group point exponentiation and pairing [Kadianakis et al.(2023)], we move them outside the circuit and implement them using Sigma. By combining the above proof systems, we can efficiently achieve malleable PoP while only introducing a minor and fixed increase in proof size and verification cost.

Contributions. We summarize the comparison of the proposed Re2creds and other ACs in Table I, which highlights the advantages of Re2creds in usability and adaptation to legacy signatures. Our main contributions are summarized as follows:

- We introduce a reusable credential presentation paradigm to significantly reduce computational overhead. Enabled by a new malleable NIZK construction called Malo, our system allows the core of a presentation to be reused across multiple authentication sessions, requiring only lightweight updates/rerandomization rather than complete regeneration.
- We propose a malleable proof of possession for widely-deployed legacy signatures. By utilizing a proof composition mechanism, the expensive cryptographic operations are moved outside the arithmetic circuit, making it practical to use credentials based on NIST-recommended curves, overcoming a major barrier to real-world adoption.
- Building upon these components, we design a reusable anonymous credential scheme in Section V. It is compatible with legacy signatures and offers efficient credential presentation via rerandomization and predicate update, greatly enhancing AC's usability. We also conduct rigorous security analysis and experimental evaluation, which fully demonstrates Re2creds's feasibility.

The rest of the paper is organized as follows. Section II introduces preliminary knowledge. Section III gives an overview of Re2creds and is followed by the introduction of building blocks in Section IV. Section V gives the concrete protocol implementation for Re2creds and Section VI summarizes the evaluation of Re2creds. We review the literature in Section VII and conclude this paper in Section VIII.

II. PRELIMINARY

In this section, we provide the definition of the notations and the cryptographic knowledge used in this paper.

A. Notations

Let $\mathbb{G}$ be a cyclic group of prime order p with additive notation, and let $\mathbb{F}_p$ denote its associated scalar field. We denote by $\mathbb{F}_p[X]$ the ring of polynomials with coefficients in $\mathbb{F}_p$. For any positive integer $n \in \mathbb{N}^+$, let $[n] := \{1, \dots, n\}$. For a set S , $|S|$ denotes its cardinality. A vector is denoted as $\vec{f} := (f_1, \dots, f_n) \in \mathbb{F}_p^n$, and the concatenation of two vectors $\vec{f}$ and $\vec{g}$ is $(\vec{f}, \vec{g})$. An NP relation $\mathcal{R}$ is defined over pairs of *public statement* x and *witness* w , such that $(x; w) \in \mathcal{R}$. We use PPT to denote probabilistic polynomial time.

B. Digital Signature

Definition 1 (Digital Signature Scheme). *A digital signature scheme is a tuple of PPT algorithms $\Sigma = (\text{Setup}, \text{KeyGen}, \text{Sign}, \text{Verify})$:*

- $\text{pp} \leftarrow \text{Setup}(1^\lambda)$: *On input a security parameter 1^λ , it outputs public parameters pp .*
- $(vk, sk) \leftarrow \text{KeyGen}(1^\lambda)$: *On input a security parameter 1^λ , it outputs a verification key vk and a secret key sk .*
- $\sigma \leftarrow \text{Sign}(sk, m)$: *On input a signing key sk and a message m , it outputs a signature σ .*
- $b \leftarrow \text{Verify}(vk, \sigma, m)$: *On input a verification key vk , a signature σ , and a message m , it outputs $b = 1$ if the signature is valid, and $b = 0$ otherwise.*

A secure digital signature scheme must satisfy *existential unforgeability under chosen message attacks (EUF-CMA)*. The formal definitions are provided in Appendix A.

C. Polynomial Commitment Scheme

In this paper, we utilize KZG10 [Kate et al.(2010)] as the instantiation of polynomial commitment schemes (PCS). Other PCSs including IPA [Bünz et al.(2018)] and FRI [Ben-Sasson et al.(2018)] can also be adopted. Following [Libert(2024)], we require the KZG scheme to satisfy *simulation extractability*, the formal definition of which is provided in Appendix B.

Definition 2 (KZG Polynomial Commitment Scheme [Kate et al.(2010)]). *The KZG polynomial commitment scheme comprises the following algorithms:*

- $(pk, vk) \leftarrow \text{Setup}(1^\lambda, D)$: *On input a security parameter 1^λ and a maximum degree D , it samples a bilinear group tuple $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_t, p, g_1, g_2, e)$ of order p . It then samples a secret $\beta \in \mathbb{F}_p$ and computes $\Sigma := (g_1, \beta g_1, \dots, \beta^D g_1) \in \mathbb{G}_1^{D+1}$. It outputs the proving key $pk := ((\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_t, p, g_1, g_2, e), \Sigma)$ and the verification key $vk := (g_1, g_2, \beta g_2)$.*
- $C \leftarrow \text{Commit}(pk, p)$: *Given a polynomial $p(X) = \sum_{j=0}^d p_j X^j \in \mathbb{F}_p[X]$ with $d \leq D$, it computes and outputs the commitment $C := \sum_{j=0}^d p_j \cdot \beta^j g_1 \in \mathbb{G}_1$.*
- $\pi \leftarrow \text{Open}(pk, p, z)$: *To open the commitment at $z \in \mathbb{F}_p$, it computes the witness polynomial $w(X) := \frac{p(X) - p(z)}{X - z} = \sum_{j=0}^{d-1} w_j X^j$ and outputs the proof $\pi := \sum_{j=0}^{d-1} w_j \cdot \beta^j g_1 \in \mathbb{G}_1$.*
- $b \leftarrow \text{Check}(vk, C, z, v, \pi)$: *It verifies the evaluation by checking if $e(C - v g_1, g_2) = e(\pi, \beta g_2 - z g_2)$. It outputs $b = 1$ if the check passes, and $b = 0$ otherwise.*

D. Proof Systems

This work mainly employs two types of proof systems: Non-interactive Zero-Knowledge Proofs (NIZK) and Sigma Protocols.

NIZKs. We present the syntax of a Non-interactive Zero Knowledge Proof in Definition 3. An NIZK is expected to satisfy *completeness*, *knowledge soundness*, and *zero-knowledge*. If an NIZK additionally satisfies *succinctness*, i.e., the verification time is bounded by $\text{poly}(\lambda, |x|, \log t)$, it is called as a zero-knowledge succinct non-interactive argument of knowledge (zk-SNARK), where λ is the security parameter, x is the statement and t is the NP verification time for x . Formal definitions are provided in Appendix D. In this paper, we mainly refer to NIZK and SNARK as those general-purpose proof systems based on circuits.

Definition 3 (NIZK). Let $\mathcal{R}$ be an NP relation. An NIZK for $\mathcal{R}$ is a tuple of polynomial-time algorithms $\Pi_{\text{NIZK}} = (\text{Setup}, \text{KeyGen}, \text{Prove}, \text{Verify})$:

- $\text{pp} \leftarrow \text{Setup}(1^\lambda)$: On input a security parameter λ , it outputs public parameters pp .
- $(\text{pk}, \text{vk}) \leftarrow \text{KeyGen}(\text{pp}, \mathcal{R})$: Given pp and relation $\mathcal{R}$, it generates a proving key pk and a verification key vk .
- $\pi \leftarrow \text{Prove}(\text{pk}, x, w)$: Given pk and a pair $(x, w) \in \mathcal{R}$, it produces a succinct proof π .
- $b \leftarrow \text{Verify}(\text{vk}, x, \pi)$: Given vk , statement x , and proof π , it outputs $b = 1$ (accept) or $b = 0$ (reject).

Sigma Protocols. A Σ -protocol [Attema and Cramer(2020)], [Xue et al.(2025)] is a three-move interactive protocol between a prover $\mathcal{P}$ and a verifier $\mathcal{V}$, defined by a tuple of PPT algorithms $\Pi_\Sigma = (\text{Setup}, \text{Prove}, \text{Verify})$. Different from circuit-based proof systems, a Sigma protocol can prove algebraic relations more efficiently, e.g., scalar multiplications over group. For simplicity, we use Fiat-Shamir heuristic to convert the interactive protocol into a non-interactive one and define its syntax $(x, w) \in \mathcal{R}$ as follows:

- $\text{pp} \leftarrow \text{Setup}(1^\lambda)$: Given security parameter λ , it outputs a common reference string pp .
- $\pi \leftarrow \text{Prove}(\text{pp}, x, w)$: Given pp and $(x, w) \in \mathcal{R}$, the prover computes a proof π and sends it to the verifier.
- $b \leftarrow \text{Verify}(\text{pp}, x, \pi)$: The verifier checks the proof π against x and outputs $b = 1$ (accept) or $b = 0$ (reject).

The adopted Sigma protocol in this work is required to be a simulation-sound non-interactive zero-knowledge (NIZK) proof of knowledge as described in [Groth(2006)].

E. Universal Composability Security

The Universal Composability (UC) framework [Canetti(2001)] provides a methodology for analyzing protocol security within complex environments.

Definition 4 (UC Security). A protocol Π UC-realizes an ideal functionality $\mathcal{F}$ if, for every PPT adversary $\mathcal{A}$ attacking the real-world execution of Π , there exists a PPT simulator $\mathcal{S}$ in the ideal-world $\mathcal{F}$ -hybrid model such that for all PPT environments $\mathcal{E}$, the following computational indistinguishability holds:

$$|\Pr[\mathcal{E}(\text{EXEC}_{\Pi, \mathcal{A}}(\lambda)) = 1] - \Pr[\mathcal{E}(\text{IDEAL}_{\mathcal{F}, \mathcal{S}}(\lambda)) = 1]| \leq \text{negl}(\lambda).$$

Key components of the framework include:

- **Ideal Functionality ($\mathcal{F}$):** A trusted party that defines desired security behaviors. It receives inputs from parties, performs computations, and delivers outputs, acting as an incorruptible mediator.
- **Realization:** Protocol Π UC-realizes $\mathcal{F}$ if no environment can distinguish between an interaction with Π and $\mathcal{A}$ (real world) versus an interaction with $\mathcal{F}$ and $\mathcal{S}$ (ideal world).

III. OVERVIEW

In this section, we start by providing an overview of Re2creds with its system and threat model. Then, we introduce the technical overview of Re2creds. Finally, we present the ideal function of Re2creds to capture the security properties of Re2creds and formalize its security definition.

A. System and Threat Model

System model. Consistent with the classic anonymous credential model, Re2creds considers an identity system comprising an issuer $\mathcal{I}$, holders $\mathcal{H}$, and verifiers $\mathcal{V}$. $\mathcal{I}$ attests the identity attributes of the holder and issues a verifiable credential. A verifiable credential can be formally represented as $\text{cred} := (\vec{a}, \sigma)$, consisting of a set of identity attribute values $\vec{a}$ and a signature σ created by $\mathcal{I}$. Specifically, σ is generated by $\mathcal{I}$'s private key sk and can be verified by its verification key vk . Once $\mathcal{H}$ receives the credential, it can generate a credential presentation for the verifier $\mathcal{V}$ who trusts the issuer with respect to its access control requirements (described by a series of attribute predicates). When receiving the presentation, $\mathcal{V}$ can verify the validity of the presentation based on vk and attribute predicates.

Threat Model. In the system, we assume that the issuer is honest and it only attests holders' attributes and does not collude with other entities. We consider malicious holders who attempt to cheat the verifier by forging a valid presentation without a valid credential from the issuer. We also consider malicious verifiers that aim to compromise the holder's privacy through presentations, such as extra credential attribute information beyond what is revealed by predicates, the holder's specific identity, or the correlation between different authentication sessions.

B. Workflow

The workflow of Re2creds comprises three core phases: credential issuance, credential presentation, and presentation verification. During credential issuance, the issuer attests attribute values $\vec{a}$ and cryptographically signs them to generate a credential. Presentation verification is executed by verifiers, who validate presentations against the issuer's verification key and predicates. These phases are usually modeled as trivial tasks, but the presentation generation process constitutes the system's efficiency bottleneck. Notably, most legacy signatures adopt the *hash-then-sign* paradigm and contain multiple group point exponentiation or pairing operations, which is inherently ZK-unfriendly and incurs substantial proof generation costs. To address these issues, we achieve efficiency gains by reusing intermediate proofs and proof composition.

Specifically, if a presentation token pt_1 exists for verifier $\mathcal{V}_1$ (session ctx_1 , predicate ϕ_1), its intermediate result π_1 can be reused. For subsequent presentations to $\mathcal{V}_2$ with the same predicate ϕ_1 (session ctx_2), the holder only needs to randomize π_1 to π_2 with lightweight computations, significantly reducing the overall cost. For $\mathcal{V}_3$ with predicates $\phi_1 \wedge \phi_2$, efficiency improves significantly by proving only additional predicate satisfaction (i.e., ϕ_2) relative to π_1 . This reuse strategy motivates our novel malleable proof system based on recursion (See Section IV-A for more details). Furthermore, directly embedding signature verification within the circuit remains costly. Decoupling computationally expensive signature verification from the circuit, while preserving holder anonymity, offers a complementary optimization path. Section IV-B details our proof composition technique with malleable NIZK addressing this challenge. Leveraging these techniques, we construct an efficient anonymous credential scheme. When generating a presentation for a list of predicates $\mathcal{T}_\phi$, the holder scans locally stored presentations for $\mathcal{T}'_\phi \subset \mathcal{T}_\phi$. The malleable proof system and proof composition are then invoked to efficiently derive the new presentation.

C. The Functionality of Re2creds

We employ the ideal functionality $\mathcal{F}_{\text{Re2creds}}$ to capture Re2creds's features as illustrated in the Definition 5 below.

Definition 5 (Ideal Functionality $\mathcal{F}_{\text{Re2creds}}$ for Re2creds). *The ideal functionality interacts with dummy parties, denoting the issuer $\mathcal{I}$, holders $\mathcal{H}$, and verifiers $\mathcal{V}$, along with an ideal-world adversary (simulator) $\mathcal{S}$. $\mathcal{F}_{\text{Re2creds}}$'s details are presented in Figure 2.*

In $\mathcal{F}_{\text{Re2creds}}$, the SETUP interface can only be activated once, and all other interfaces ignore all input until a SETUP message has been completed. Besides, when conducting a check, $\mathcal{F}_{\text{Re2creds}}$ aborts with the output $\perp$ to the caller if the check fails. $\mathcal{F}_{\text{Re2creds}}$ is designed to ensure the following security properties:

- **Credential privacy:** The verifier cannot obtain extra information about the presented credential cred beyond the fact that $\vec{a}_i$ satisfies all the predicates listed in $\mathcal{T}_\phi$ because the presentation token generated by Present does not take a specific credential's attributes $\vec{a}_i$ as input.
- **Identity privacy:** The verifier cannot identify the specific holder it interacts with. Similar to *credential privacy*, Present takes no holder-specific data as input, and the presentation tokens from different holders share the same distribution to hide the identity.
- **Unlinkability:** The malicious verifier cannot identify presentations from the same credential. Similar to *credential/identity privacy*, it can also be directly acquired by the construction of the algorithm Present.
- **Unforgeability:** The holder cannot generate a valid presentation without a valid credential. The PRESENT interface first checks the holder's issuance record and predicates on the attribute values, ensuring that the algorithm Present will not be invoked if any condition is not satisfied.

These security properties are provided by the ideal function $\mathcal{F}_{\text{Re2creds}}$ and a secure implementation of $\mathcal{F}_{\text{Re2creds}}$ in the protocol Π_{Re2creds} also inherently preserves these properties in the real-world deployment. When implementing Π_{Re2creds} , Re2creds uses three ideal functionalities: $\mathcal{F}_{\text{crs}}$, $\mathcal{F}_{\text{ca}}$, $\mathcal{F}_{\text{smt}}$ defined in [Canetti(2000)], [Canetti(2004)].

Formal security definition. Consider a protocol Π_{Re2creds} that interfaces with $\mathcal{F}_{\text{crs}}$, $\mathcal{F}_{\text{ca}}$, and $\mathcal{F}_{\text{smt}}$. When an environment $\mathcal{E}$ interacts with Π_{Re2creds} and a real-world adversary $\mathcal{A}$, given security parameter $\lambda \in \mathbb{N}$ and auxiliary input $z \in \{0, 1\}^*$, the resulting output is denoted as $\text{EXEC}_{\Pi_{\text{Re2creds}}, \mathcal{A}, \mathcal{E}}^{\mathcal{F}_{\text{crs}}, \mathcal{F}_{\text{ca}}, \mathcal{F}_{\text{smt}}}(\lambda, z)$. Correspondingly, when $\mathcal{E}$ interacts with the ideal functionality $\mathcal{F}_{\text{Re2creds}}$ and an ideal-world adversary $\mathcal{S}$, the output is represented as $\text{IDEAL}_{\mathcal{F}_{\text{Re2creds}}, \mathcal{S}, \mathcal{E}}(\lambda, z)$.

Here, we formally define the security properties of Re2creds with the protocol specification Π_{Re2creds} illustrated in Section V.

Functionality $\mathcal{F}_{\text{Re2creds}}$

On input (SETUP, sid) from $\mathcal{I}$:

- 1 : Check that $\text{sid} = (\mathcal{I}, \text{sid}')$.
- 2 : Output (SETUP, sid) to $\mathcal{S}$ and wait for response
(SETUP, sid, Present, Verify, $\vec{\mathbb{A}}$) from $\mathcal{S}$.
- 3 : Store algorithms Present and Verify and credential
attribute value space $\vec{\mathbb{A}}$.
- 4 : Set $\mathcal{L}_{\text{pt}} := \emptyset$ and $\mathcal{L}_c := \emptyset$.
- 5 : Output (SETUPDONE, sid) to $\mathcal{I}$.

On input (ISSUE, sid, $\vec{a}_i$, $\mathcal{H}_i$) from $\mathcal{I}$ with $\vec{a}_i \in \vec{\mathbb{A}}$.

- 6 : Check $\text{sid} = (\mathcal{I}, \text{sid}')$, output (ALLOWISSUE, sid, $\mathcal{H}_i$)
to $\mathcal{S}$ and wait for input (ALLOWISSUE, sid) from $\mathcal{S}$.
- 7 : Add an entry $(\mathcal{H}_i, \vec{a}_i)$ to $\mathcal{L}_c$.
- 8 : Output (ISSUE, sid, $\vec{a}_i$) to $\mathcal{H}_i$.

On input (PRESENT, sid, $\mathcal{T}_\phi$, ctx) from a holder $\mathcal{H}_i$.

- 9 : Check that an entry $(\mathcal{H}_i, \vec{a}_i)$ exists in $\mathcal{L}_c$ and
 $\phi(\vec{a}_i) = 1$ for each $\phi \in \mathcal{T}_\phi$.
- 10 : Set $\text{pt} \leftarrow \text{Present}(\text{ctx}, \mathcal{T}_\phi)$, abort if $\text{Verify}(\text{pt}, \mathcal{T}_\phi, \text{ctx}) = 0$.
- 11 : Record $(\mathcal{H}_i, \mathcal{T}_\phi, \text{ctx}, \text{pt})$ in $\mathcal{L}_{\text{pt}}$ and output (TOKEN,
sid, pt) to $\mathcal{H}_i$.

On input (Verify, sid, ctx, pt, $\mathcal{T}_\phi$) from a verifier $\mathcal{V}$.

- 12 : If there is no record $(\cdot, \mathcal{T}_\phi, \text{ctx}, \text{pt})$ in $\mathcal{L}_{\text{pt}}$, $\mathcal{I}$ is honest,
and there is no corrupt $\mathcal{H}_j$ such that $(\mathcal{H}_j, \vec{a}_j) \in \mathcal{L}_c$
with $\phi(\vec{a}_j) = 1$ for all $\phi \in \mathcal{T}_\phi$, set $b := 0$.
- 13 : Else, set $b := \text{Verify}(\text{pt}, \mathcal{T}_\phi, \text{ctx})$.
- 14 : Output (VERIFIED, sid, b) to $\mathcal{H}_i$.

Fig. 2: Ideal functionality $\mathcal{F}_{\text{Re2creds}}$.

Definition 6 (Re2creds Security). *Let $\lambda \in \mathbb{N}$ denote a security parameter, and let Π_{Re2creds} represent a protocol operating within the $(\mathcal{F}_{\text{crs}}, \mathcal{F}_{\text{ca}}, \mathcal{F}_{\text{smt}})$ -hybrid model. We define that Π_{Re2creds} securely realizes $\mathcal{F}_{\text{Re2creds}}$ under the $(\mathcal{F}_{\text{crs}}, \mathcal{F}_{\text{ca}}, \mathcal{F}_{\text{smt}})$ -hybrid world if for all PPT real-world adversary $\mathcal{A}$ engaging with Π_{Re2creds} , there exists a PPT ideal-world adversary $\mathcal{S}$ interacting with $\mathcal{F}_{\text{Re2creds}}$ such that the following holds for all PPT environments $\mathcal{E}$ and for all auxiliary inputs $z \in \{0, 1\}^*$:*

$$\text{EXEC}_{\Pi_{\text{Re2creds}}, \mathcal{A}, \mathcal{E}}^{\mathcal{F}_{\text{crs}}, \mathcal{F}_{\text{ca}}, \mathcal{F}_{\text{smt}}}(\lambda, z) \approx \text{IDEAL}_{\mathcal{F}_{\text{Re2creds}}, \mathcal{S}, \mathcal{E}}(\lambda, z).$$

IV. BUILDING BLOCKS

This section introduces the foundational building blocks for our proposed system. We first present the construction of a Malleable NIZK based on recursive arguments in Section IV-A. This serves as the underlying proof system for our anonymous credential scheme, providing essential features such as fast proving time, constant proof size, and privacy. Subsequently, in Section IV-B, we propose a novel Malleable Proof of Possession (PoP) for signatures which leverages proof composition to efficiently prove knowledge of a signature while supporting malleability.

A. Malleable NIZK Construction

In this subsection, we describe the concrete instantiation of Malo, a malleable NIZK based on recursive arguments. While Chakraborty et al. [Chakraborty et al.(2025)] provided a theoretical result indicating that a malleable NIZK can be constructed from a recursive one, practical instantiations remain underexplored. Moreover, to satisfy the requirements of our anonymous credential scheme, the candidate recursive NIZK construction is expected to meet the following criteria:

- Constant proof size: for succinct verification of credential presentations, Re2creds should be compiled with a constant-sized polynomial commitment scheme, such as KZG10 [Kate et al.(2010)].
- Fast proving time: different from other applications of recursive NIZKs that prioritize minimizing verification time, anonymous credential system requires efficient proof generation to ensure fast credential presentations. Hence we prefer the

Halo-style recursive construction [Bowe et al.(2019)], [Boneh et al.(2021)] rather than Nova-style [Kothapalli et al.(2022)], [Kothapalli and Setty(2022)], [Zheng et al.(2023)].

- Non-uniform circuits: to allow the holder to prove different predicates of the credentials, the underlying recursive NIZK is required to switch the circuit at each recursion, which can be achieved by the construction in [Kothapalli and Setty(2022)].
- Zero knowledge: to ensure the privacy of presentation proofs, the malleable NIZK should provide derivation privacy, which depends on the zero-knowledge property of the underlying recursive argument.

Unfortunately, existing recursive NIZKs fall short of meeting these criteria simultaneously. Therefore, we propose a concrete construction for the malleable NIZK built from a new recursive NIZK improved on existing solutions.

1) *Malleable NIZK*: We first recap the definition of malleable NIZKs [Chase et al.(2012)], [Chakraborty et al.(2025)]. We begin by defining an n -ary transformation admissible for an NP relation $\mathcal{R}$ as $\mathcal{T}$. It implies that if a set of instance-witness pairs $\{(x_i, w_i)\}_{i \in [n]}$ satisfies $\mathcal{R}$, i.e., $\forall i \in [n] : (x_i, w_i) \in \mathcal{R}$, then the transformed pair $(x, w) := \mathcal{T}(\{(x_i, w_i)\}_{i \in [n]})$ also satisfies $\mathcal{R}$. Then, we define the malleable NIZK as follows:

Definition 7. A non-interactive zero-knowledge proof system $\text{NIZK} = (\text{Setup}, \text{Prove}, \text{Verify})$ for $\mathcal{R}$ is **malleable** with respect to a set of transformations $\vec{\mathcal{T}}$, if there exists a PPT algorithm ZKEval that takes as input the public parameters pp , an n -ary transformation $\mathcal{T} \in \vec{\mathcal{T}}$, and n statements with corresponding proofs $\{(x_i, \pi_i)\}_{i \in [n]}$, and outputs a valid proof π for the transformed statement-witness pair $(x, w) := \mathcal{T}(\{(x_i, w_i)\}_{i \in [n]})$.

Typically, a malleable NIZK is assumed to satisfy the following security properties:

- *Soundness*: For any malicious prover who outputs a valid proof, there exists an extractor that can extract the corresponding witness. Chakraborty et al. [Chakraborty et al.(2025)] introduced a relaxed notion of controlled-malleable simulation extractability; the formal definition is provided in Definition 16 in Appendix C.
- *Derivation privacy*: Any PPT adversary cannot distinguish the output of ZKEval from a freshly generated proof for the same transformed statement. A weaker notion known as leveled derivation privacy is also mentioned for bootstrapping, where proof leaks the number of applied transformations.

2) *Toolbox*: We adopt the framework used in Halo and its variants [Bowe et al.(2019)], [Boneh et al.(2021)], [Bünz et al.(2021)], which integrates a committed relation, a polynomial commitment aggregation scheme, and a cycle of curves to construct an efficient recursive SNARK.

a) *Plonkish relations*: We first review the relations in the Plonk proof system [Gabizon et al.(2019)]. Unlike Interactive Oracle Proof (IOP) systems based on R1CS, such as Groth16 [Groth(2016)] or Marlin [Chiesa et al.(2020)], Plonk offers better expressiveness by supporting specific custom gates and lookup operations. Specifically, the prover in a Plonk IOP first arithmetizes the computation trace of a circuit into a polynomial equation interpolated from the left inputs, right inputs, and outputs of all gates. To prove that the equation holds for all points over a cyclic subgroup $H \in \mathbb{F}_p$, the prover and verifier engage in a series of interactions, detailed in Appendix F. After the interaction, the original polynomial equation holding on the subgroup H is reduced to a relation holding on several randomly sampled points with negligible soundness error. Without loss of generality, we can apply a polynomial commitment to compile all the polynomial oracles into a vector of polynomial commitments denoted as $f_1, f_2, \dots, f_k$, where each f_i for $i \in [k]$ needs to be opened at a point z_i . The derived committed plonkish relation with respect to a tuple of algorithms $\text{NIZK}_{\text{plonk}} := (\text{Prove}, \text{Verify})$ is given below:

Definition 8 (Committed Plonkish Relation). Consider a finite field $\mathbb{F}_p$ and a polynomial commitment scheme PCS (e.g., KZG10 [Kate et al.(2010)]). Let public parameters pp consist of the degree bound $d \in \mathbb{N}$, and commitment parameters pp_{PCS} . The committed plonkish relation is an NP relation denoted as $\mathcal{R}_{\text{plonk}}$ with respect to pp , where a witness $w = (f_1, \dots, f_k)$ for a statements $x = (C_1, \dots, C_k, z_1, \dots, z_k, v_1, \dots, v_k)$ satisfies $\mathcal{R}_{\text{plonk}}$ if (1) $f_i(z_i) = v_i$, (2) $C_i = \text{PCS.Commit}(\text{pk}, f_i)$ for all $i \in [k]$, and (3) $\deg(f_i) \leq d$ for all $i \in [k]$.

b) *PCS Aggregation*: In this part, we adopt the definition of PCS aggregation scheme given in [Boneh et al.(2021)], which is the core technique for recursive constructions. Similar to signature aggregation, the PCS aggregation takes inputs as k PCS instance tuples $(C_1, z_1, v_1), \dots, (C_k, z_k, v_k)$ and outputs an aggregated one as (C^*, z^*, v^*) such that applying Open to open $C^* \in \mathbb{G}$ at point $z^* \in \mathbb{F}_p$ to $v^* \in \mathbb{F}_p$ suffices to open each $C_i \in \mathbb{G}$ at $z_i \in \mathbb{F}_p$ to $v_i \in \mathbb{F}_p$ for $i \in [k]$. We formalize this scheme in Definition 9. Note that in this paper, we utilize KZG10 [Kate et al.(2010)] as the underlying PCS and adopt the aggregation protocol proposed in [Boneh et al.(2021)]. We detail the aggregation protocol in Figure 6 in Appendix B.

Definition 9 (PCS Aggregation [Boneh et al.(2021)]). Let $\text{PCS} = (\text{Setup}, \text{Commit}, \text{Open}, \text{Check})$ denote a polynomial commitment scheme over group $\mathbb{G}$. An aggregation scheme for PCS is a public-coin interactive protocol Aggregate with public inputs $\vec{C} = (C_1, \dots, C_k) \in \mathbb{G}^k$, $\vec{z} \in \mathbb{F}_p^k$, $\vec{v} \in \mathbb{F}_p^k$, and private inputs $\vec{f} \in \mathbb{F}_p^{(<d)}[X]^k$ such that $f_i(z_i) = v_i \wedge C_i = \text{Commit}(f_i)$ for all $i \in [k]$:

$$((f^*; C^*, z^*, v^*), \pi_{\text{agg}}) \leftarrow \text{Aggregate}(\mathcal{P}(\vec{f}, \vec{z}), \mathcal{V}(\vec{C}, \vec{z}, \vec{v})),$$

where f^* satisfies $f^*(z^*) = v^* \wedge C^* = \text{Commit}(f^*)$, and π_{agg} is an aggregation proof.

c) *Cycle of curves*: Building a recursive argument necessitates a prover to represent the logic of “verifying” or, at least, “aggregating” the proof output from the previous step as a circuit, and generate a proof for this circuit. This construction inevitably incurs significant costs for representing group operations in circuits, which are common in verification and aggregation algorithms. To mitigate the cost of group operations, the implementations of most recursive arguments rely on a cycle of elliptic curves, e.g., BN254/Grumpkin [Explorations([n.d.])]. Concretely, denote an elliptic curve group as $\mathbb{G}$, where the scalar field of $\mathbb{G}$ is referred to as a field $\mathbb{F}$ with order $|\mathbb{G}|$, and the base field of $\mathbb{G}$ as another field $\mathbb{F}'$ over which the elliptic curve is defined. For two groups $\mathbb{G}_1, \mathbb{G}_2$ of two elliptic curves, if group $\mathbb{G}_1$ has scalar and base fields $\mathbb{F}_1, \mathbb{F}_2$, and group $\mathbb{G}_2$ has scalar and base fields $\mathbb{F}_2, \mathbb{F}_1$, then we say the two curves form a cycle of elliptic curves. As a result, the group operations in $\mathbb{G}_1$ can be efficiently represented over the base field $\mathbb{F}_2$ (the scalar field of $\mathbb{G}_2$), and vice versa.

3) *The Construction of Malo*: Now, we are ready to build a malleable NIZK for the Plonkish constraint system based on the KZG scheme [Kate et al.(2010)]. We first omit the succinctness and derivation privacy of the malleable NIZK to accomodate the framework in [Chakraborty et al.(2025)], and then discuss how to achieve these properties later. To allow multiple transformations $\vec{T}$, we adopt the technique from SuperNova [Kothapalli and Setty(2022)], where the prover maintains a list of all intermediate (non-succinct) committed Plonkish pairs, i.e., $(\vec{X}, \vec{W})$.

Construction 1 (Malo: Malleable Halo-Style NIZK). *Let $\vec{T} = (\mathcal{T}_1, \dots, \mathcal{T}_\ell)$ denote a collection of polynomial-time predicates (i.e., transformations), and $(x_\perp, w_\perp) \in \mathcal{R}_{\text{cm-plonk}}$ denote a trivial committed Plonkish instance-witness pair for any of these predicates. A malleable NIZK is a tuple of five algorithms: $\Pi = (\text{Setup}, \text{KeyGen}, \text{Prove}, \text{ZKEval}, \text{Verify})$ as below:*

- $\Pi.\text{Setup}(1^\lambda) \rightarrow \text{pp}$: *the setup algorithm takes the security parameters and outputs public parameters pp.*
- $\Pi.\text{KeyGen}(\text{pp}, \vec{T}) \rightarrow (\text{pk}, \text{vk})$: *it computes $(\text{pk}_i, \text{vk}_i), i \in [\ell]$ for each committed Plonkish relation representing $\mathcal{T}_i$. Then it outputs the proving/verification keys as $(\text{pk}, \text{vk}) := ((\mathcal{T}_i, \text{pk}_i)_{i=1}^\ell, (\mathcal{T}_i, \text{vk}_i)_{i=1}^\ell)$.*
- $\Pi.\text{Prove}(\text{pk}, x_0, w_0) \rightarrow (x_1, \pi_1)$:
 - *it parses x_0 as $(0, z_0)$, check that $z_0 = w_0 = \perp$ and set $z_1 = z_0$;*
 - *it initializes $(\vec{X}_1, \vec{W}_1) := (\{\vec{x}_\perp\}_{i=1}^\ell, \{\vec{w}_\perp\}_{i=1}^\ell)$, $(x_1, w_1) := (x_\perp, w_\perp)$;*
 - *it outputs $(x_1, \pi_1) := ((1, z_0, z_1), ((\vec{X}_1, \vec{W}_1), (x_1, w_1)))$.*
- $\Pi.\text{ZKEval}(\text{pk}, \mathcal{T}_i \in \vec{T}, (x_i, \pi_i)) \rightarrow (x_{i+1}, \pi_{i+1})$:
 - *it parses x_i as (i, z_i) , and π_i as $((\vec{X}_i, \vec{W}_i), (x_i, w_i))$;*
 - *it computes the counter $\text{pc} \in [\ell]$ according to i and set $w_i := ((\vec{X}_i[\text{pc}], \vec{W}_i[\text{pc}]), (x_i, w_i))$;*
 - *it computes $(x_{i+1}, w_{i+1}) \leftarrow \mathcal{T}_i(x_i, w_i)$ by the following steps:*
 - * *compute z_{i+1} from z_i as per to $\mathcal{T}_i$, and set $x_{i+1} := (i+1, z_0, z_{i+1})$;*
 - * *run $(\vec{X}_{i+1}, \vec{W}_{i+1}, \pi_{\text{agg}}) \leftarrow \text{KZG.Aggregate}((\vec{X}_i[\text{pc}], \vec{W}_i[\text{pc}]), (\vec{x}_i[\text{pc}], \vec{w}_i[\text{pc}]))$;*
 - * *run $(x_{i+1}, w_{i+1}) \leftarrow \text{NIZK}_{\text{plonk}}.\text{Prove}(\text{pk}, (i, z_0, z_i), w_i, \pi_{\text{agg}})$;*
 - * *update the pc-th element in $(\vec{X}_i, \vec{W}_i)$ with $(\vec{X}_{i+1}, \vec{W}_{i+1})$ and set $\pi_{i+1} := ((\vec{X}_{i+1}, \vec{W}_{i+1}), (x_{i+1}, w_{i+1}))$.*
 - *it outputs (x_{i+1}, π_{i+1}) .*
- $\Pi.\text{Verify}(\text{vk}, x_i, \pi_i) \rightarrow b$:
 - *it parses x_i as (i, z_i) , and π_i as $((\vec{X}_i, \vec{W}_i), (x_i, w_i))$;*
 - *it checks that (x_i, w_i) is a satisfying instance-witness pair by running $b_0 \leftarrow \text{NIZK}_{\text{plonk}}.\text{Verify}(\text{vk}, (i, z_0, z_i), x_i, w_i)$.*
 - *it checks that $(\vec{X}_i[j], \vec{W}_i[j])$ is a satisfying instance-witness pair for $j \in [\ell]$ by running $b_j \leftarrow \text{NIZK}_{\text{plonk}}.\text{Verify}(\text{vk}, \vec{X}_i[j], \vec{W}_i[j])$.*
 - *it outputs $b := \bigwedge_{j=0}^\ell b_j$.*

Lemma 1 (Malleable NIZK Security). *The malleable NIZK system Π in Construction 1 satisfies completeness. It further satisfies controlled-malleable simulation extractability in the random oracle model if the underlying polynomial commitment scheme and the $\text{NIZK}_{\text{plonk}}$ for the committed Plonkish relation provides simulation extractability. It satisfies levelled derivation privacy if the underlying KZG provides hiding and the underlying $\text{NIZK}_{\text{plonk}}$ provides zero-knowledge.*

The formal proof generally follows [Chakraborty et al.(2025)] and is deferred to the full version due to page limits. So far, we obtain a malleable NARK that supports multiple predicates and levelled derivation privacy. However, it is still insufficient to achieve the requirement for anonymous credential scheme since (1) the verification cost linearly depends on the number of applied transformations, violating succinctness and (2) the derivation privacy is weak as it leaks the pattern of applied transformations. To solve the first issue, we further introduce two auxiliary algorithms $\text{KZG.auxProve}, \text{KZG.auxVerify}$ that achieves succinctness.

- $\Pi.\text{auxProve}(\text{pk}, x_i, \pi_i) \rightarrow \text{pf}_i$:
 - *it parses π_i as committed Plonkish instance-witness pairs $((\vec{X}_i, \vec{W}_i), (x_i, w_i))$;*
 - *it runs $((x^*, w^*), \pi_{\text{agg}}) \leftarrow \text{KZG.Aggregate}((\vec{X}_i, \vec{W}_i), (x_i, w_i))$;*
 - *it generates the evaluation proof π_{eval} for commitment $C \in x^*$ and polynomial $f(X) \in w^*$, by running $\pi_{\text{eval}} \leftarrow \text{KZG.Open}(\text{pk}, z, f)$, where z is derived from a random oracle;*

- it outputs the succinct proof as $\text{pf}_i = (\vec{X}_i, x_i, x^*, \pi_{\text{agg}}, \pi_{\text{eval}})$.
- $\Pi.\text{auxVerify}(vk, x_i, \text{pf}_i)$
 - it parses x_i as (i, z_0, z_i) and pf_i as $(\vec{X}_i, x_i, x^*, \pi_{\text{agg}}, \pi_{\text{eval}})$;
 - it checks $\pi_{\text{agg}}, \pi_{\text{eval}}$ accordingly.

We can reduce the verification cost to a constant size by allowing the prover at the i -th step to aggregate all instance-witness pairs $((\vec{X}_i, \vec{W}_i), (x_i, w_i))$ into a single committed Plonkish pair (x^*, w^*) . Then the prover computes a single evaluation proof π_{eval} for the aggregated instance x^* . By further utilizing the adversarial one-way function proposed in [Chakraborty et al.(2025)] to generate the index i , we can achieve a stronger derivation privacy as defined in Definition 23.

Remark 1. Strictly speaking, the above SNARK is not malleable since there does not exist an ZKEval algorithm outputs the succinct proof pf_{i+1} from a previous one pf_i without knowing the witness in π_i . Owing to this reason, we build our Anonymous Credential scheme based on the malleable NIZK directly and additionally calls the algorithms $(\text{auxProve}, \text{auxVerify})$ for succinctness, which is proved to be secure in Appendix F.

B. Malleable PoP Scheme

We further discuss another technical bottleneck in constructing an efficient anonymous credential scheme: how to efficiently prove the possession of a valid signature on messages. We elaborate on this problem by a naive solution in Section IV-B1 and then present our optimized construction in Section IV-B2.

1) *Strawman Solution:* Given a signature tuple (vk, σ, m) , there exists a straightforward construction for proving the possession of a valid signature, as formalized below:

Definition 10 (Proof of Possession for Signatures [Woo et al.(2025)]). *Given a signature scheme $\text{Sig} = (\text{Setup}, \text{KeyGen}, \text{Sign}, \text{Verify})$, assume the signature σ on a message m with respect to verification key vk is honestly generated by $\text{Sig.Sign}(sk, m)$. Then we define a proof of possession for signature σ with the relation:*

$$\mathcal{R}_{\text{PoP}} := \{ (vk; m, \sigma) : \text{Sig.Verify}(vk, \sigma, m) = 1 \}.$$

Typically, to prove the above $\mathcal{R}_{\text{PoP}}$ using a general-purpose proof system, existing solutions execute the proving algorithm on a circuit directly instantiated from the verification algorithm Sig.Verify . If the underlying proof system supports *malleability*, the prover can simply modify the generated proof using an admissible transformation T instead of regenerating it from the original signature, yielding a malleable PoP scheme as described below.

Construction 2 (Malleable PoP). *The construction utilizes a signature scheme $\text{Sig} = (\text{Setup}, \text{KeyGen}, \text{Sign}, \text{Verify})$ and a malleable NIZK $\Pi = (\text{Setup}, \text{Prove}, \text{Verify}, \text{ZKEval})$ which supports an admissible set of transformations $\vec{T}$. Assume the signature σ on a message m is validly generated via $\text{Sig.Sign}(sk, m)$. Assume there exists a transformation $T \in \vec{T}$ that transforms the instance $(vk; m, \sigma)$ to $(vk; m', \sigma')$. The construction for a malleable PoP for signatures proceeds as follows:*

- *Prove:* The prover runs $\pi \leftarrow \Pi.\text{Prove}(\text{pk}, x, w)$ for the relation $\mathcal{R}_{\text{PoP}}$, with statement $x = vk$ and witness $w = (m, \sigma)$.
- *Derive:* The prover runs $\Pi.\text{ZKEval}(\text{pp}, T, (x, \pi))$ to transform the proof π into a new proof π' verifying the transformed statement-witness $(vk; m', \sigma')$.
- *Verify:* Upon receiving the proof π for $\mathcal{R}_{\text{PoP}}$, the verifier executes $b \leftarrow \Pi.\text{Verify}(\text{pp}, x, \pi)$.

Remark 2 (Efficiency Problem). *Although the above technique ensures the confidentiality of m and σ , and avoids the computational overhead of repeatedly generating fresh proofs to achieve unlinkability (preventing an attacker from linking two presentations to the same signature), it faces a serious efficiency challenge. Most signature verification algorithms involve complex operations, such as group point multi-exponentiations and bilinear pairings, which are prohibitively expensive to evaluate within the arithmetic circuits of current NIZKs. We list the estimated circuit constraints for curve operations in Table II, including a zk-friendly curve Baby Jubjub, a normal curve P-256 used in legacy signatures, and a pairing-enabled curve BN254.*

TABLE II: Circuit constraints for operations on curves.

Curves	Point Addition	Point Exponentiation	Pairing
Baby Jubjub	$\approx 2^3$	$\approx 2^{11}$	-
P-256	$\approx 2^9$	$\approx 2^{18}$	-
BN254	$\approx 2^9$	$\approx 2^{18}$	$> 2^{20}$

¹ Operations over BN254 are evaluated in $\mathbb{G}_1$.

² Utilize lookup tables to optimize circuits [Woo et al.(2025)].

2) *Our PoP Construction*: To address the aforementioned efficiency issue, we leverage proof composition [Campanelli et al.(2019)] and a recursive-based malleable NIZK. Proof composition enables the combination of proofs from multiple systems, allowing for the optimal choice of proof system for each subroutine in a heterogeneous computation. For simplicity, we assume the signature scheme satisfies the following two properties:

- The verification algorithm can be split into two consecutive processes: $V_1(vk, \sigma, m) \rightarrow (c, \text{aux})$ and $V_2(vk, \sigma, c, \text{aux}) \rightarrow \{0, 1\}$, where c is the hash image and aux denotes auxiliary information from the intermediate computation.
- There exists a tuple of admissible transformation $(T_\Sigma, T_{\text{rand}})$ from (σ, c, aux) to $(\sigma', c', \text{aux}')$ via

$$\sigma' := T_\Sigma(\sigma; r_s), (c', \text{aux}') := T_{\text{rand}}(c, \text{aux}; r_c),$$

where $T_\Sigma, T_{\text{rand}}$ are parameterized by random inputs r_s, r_c , respectively. The transformed tuple $(\sigma', c', \text{aux}')$ satisfies $V'_2(vk, \sigma', c', \text{aux}', r_s, r_c) = 1$, where V'_2 is a relaxed verification algorithm with additional secret inputs r_s and r_c . Furthermore, we require that $(\sigma', c', \text{aux}')$ is indistinguishable from (σ, c, aux) to ensure derivation privacy.

Typically, V_1 represents operations that can be easily encoded into arithmetic circuits, while V_2 contains circuit-unfriendly operations like pairings or group point exponentiations. Our observation is that the prover can only instantiate V_1 within the NIZK circuit, defined as $\mathcal{R}_{V_1} = \{(vk, c, \text{aux}; \sigma, m) : (c, \text{aux}) = V_1(vk, \sigma, m)\}$. As for V_2 , the prover invokes a Sigma protocol instead, where the relation is $\mathcal{R}_{V_2} = \{(vk, \sigma, c, \text{aux}) : V_2(vk, \sigma, c, \text{aux}) = 1\}$.

The key innovation lies in the proof derivation (randomization). According to Construction 2, the prover first computes σ' and (c', aux') by invoking $T_\Sigma(\sigma; r_s)$ and $T_{\text{rand}}(c, \text{aux}; r_c)$, respectively. Then it derives new proofs $\pi'_\Sigma, \pi'_{\text{NIZK}}$ in the following two steps.

- The prover directly generates a fresh proof π'_Σ from the scratch, for $\mathcal{R}_{V'_2}$ as define below

$$\mathcal{R}_{V'_2} := \left\{ \left(\begin{array}{c} vk, \sigma', c', \text{aux}'; \\ \sigma, r_s, r_c \end{array} \right) : \begin{array}{l} \sigma' = T_\Sigma(\sigma; r_s) \wedge \\ 1 = V'_2(vk, \sigma', c', \text{aux}', r_s, r_c) \end{array} \right\}.$$

- The prover derives a new proof π'_{NIZK} by invoking the $\Pi.\text{ZKEval}(\text{pp}, \mathcal{T}_{\text{rand}}, (x, \pi_{\text{NIZK}}))$, where the $\mathcal{T}_{\text{rand}}$ is derived from T_{rand} according to Construction 1. We denote $\mathcal{T}_{\text{rand}}$ below as an *augmented transformation* following the conventions in [Kothapalli et al.(2022)], which outputs a new statement-witness pair (x', w') from the original (x, w) :

$$\begin{aligned} x' &:= (\hat{\mathbf{i}}, (\hat{c}, \hat{\text{aux}}), T_{\text{rand}}(c, \text{aux}; r_c)), \\ w' &:= ((\vec{X}_{\hat{\mathbf{i}}}[\text{pc}], \vec{W}_{\hat{\mathbf{i}}}[\text{pc}]), (x_{\hat{\mathbf{i}}}, w_{\hat{\mathbf{i}}})), \end{aligned}$$

where $\hat{c}, \hat{\text{aux}}$ is the initial hash image and auxiliary information, and $\hat{\mathbf{i}}, (\vec{X}, \vec{W}), (x, w), \text{pc}$ are defined in Construction 1.

Construction 3 (Malleable PoP from Proof Composition). *There is a signature scheme $\text{Sig} = (\text{Setup}, \text{KeyGen}, \text{Sign}, \text{Verify})$, a malleable NIZK $\Pi = (\text{Setup}, \text{KeyGen}, \text{Prove}, \text{Verify}, \text{ZKEval})$ for an admissible set of transformations $\vec{T}$, and a Sigma protocol $\Pi_\Sigma = (\text{Setup}, \text{Prove}, \text{Verify})$. Assume the signature σ on a message m is honestly generated by $\text{Sig}.\text{Sign}(sk, m)$. Assume there exists a transformation $T = (T_\Sigma, T_{\text{rand}})$ to transform the public inputs (c, aux, σ) to $(c', \text{aux}', \sigma')$. A malleable PoP consists of the following:*

- *Prove*: the prover runs the malleable NIZK proving algorithm $\pi_\Sigma \leftarrow \Pi.\text{Prove}(\text{pk}, (vk, c, \text{aux}), (\sigma, m))$ for relation $\mathcal{R}_{V_1}$ and the Sigma proving algorithm $\pi_{\text{NIZK}} \leftarrow \Pi_\Sigma.\text{Prove}(\text{pp}, (vk, \sigma, c, \text{aux}), (\sigma, \perp, \perp))$ for relation $\mathcal{R}_{V'_2}$. The output proof is $\pi := (\pi_\Sigma, \pi_{\text{NIZK}})$.
- *Derive*: to derive a new proof from π , it first computes $(\sigma', c', \text{aux}')$ via T_Σ and T_{rand} , then invokes:
 - $((c', \text{aux}'), \pi'_{\text{NIZK}}) \leftarrow \Pi.\text{ZKEval}(\text{pk}, \mathcal{T}_{\text{rand}}, ((c, \text{aux}), \pi_{\text{NIZK}}))$;
 - $\pi'_\Sigma \leftarrow \Pi_\Sigma.\text{Prove}(\text{pp}, (vk, c', \sigma', \text{aux}'), (\sigma, r_s, r_c))$ for relation $\mathcal{R}_{V'_2}$.
 It outputs the new proof $\pi' := (\pi'_\Sigma, \pi'_{\text{NIZK}})$.
- *Verify*: upon receiving the composed proof π and public inputs $(vk, \sigma, c, \text{aux})$, the verifier runs the algorithm $b_1 \leftarrow \Pi.\text{Verify}(vk, (c, \text{aux}), \pi_{\text{NIZK}})$ and $b_2 \leftarrow \Pi_\Sigma.\text{Verify}(\text{pp}, (vk, \sigma, c, \text{aux}), \pi_\Sigma)$.

To ensure the linking of the shared witness between the two proof systems, we implicitly apply the technique from [Kadianakis et al.(2023)], which requires an extra hash computation in the circuit for each shared element. Informally, Construction 3 satisfies the security properties of PoPs, provided that (1) the signature scheme $\text{Sig} = (\text{Setup}, \text{KeyGen}, \text{Sign}, \text{Verify})$ satisfies simulation-unforgeability; (2) the malleable NIZK $\Pi = (\text{Setup}, \text{KeyGen}, \text{Prove}, \text{Verify}, \text{ZKEval})$ satisfies controlled-malleable simulation-extractability and relaxed derivation privacy for an admissible set of transformations $\vec{T}$ including $\mathcal{T}_{\text{rand}}$; and (3) the transformed outputs $(c', \sigma', \text{aux}')$ are indistinguishable from the original (c, σ, aux) .

To demonstrate the versatility of our construction, we a set of instantiations of existing legacy signatures, including BLS [Boneh et al.(2019)], ECDSA [Vanstone(1992)], RSA [Damgård and Koprowski(2001)], and Schnorr [Schnorr(1991)] in Appendix F.

V. Re2creds: ANONYMOUS CREDENTIAL SYSTEM

We first introduce new definitions for predicates used in anonymous credential schemes in Section V-A. Then, based on the introduced building blocks and predicates, we describe the full scheme for $\mathcal{F}_{\text{Re2creds}}$ in Section V-B.

A. Predicate Definitions

When instantiating Re2creds, we convert basic operations into the language of malleable NIZK systems. Specifically, we abstract the operations a holder performs on a credential over time as an incremental computation problem in chronological order. For example, assuming a holder generates a simple PoP at step 1, updates the credential attribute predicate at step 2, and randomizes the proof at step 3, we can model this as the computation $\pi = T_3(T_2(T_1(m, \sigma, vk)))$, where each transformation function T_i is selected from an admissible set $\bar{T} = \{T_{PoP}, T_{rand}, T_{upd}\}$.

Definition 11 (PoP for credentials). *To generate a PoP for a message m signed by an issuer with key pair (sk, vk) , the prover can apply a malleable NIZK Π , a sigma protocol Π_Σ , and a signature scheme Sig in terms of a transformation $T := (\perp, T_{PoP})$. The result is a presentation token $\text{pt} = (\sigma, c, \text{aux}, \pi_\Sigma, \pi_{\text{NIZK}})$ computed via the following steps by the holder:*

- obtains a signature $\sigma \leftarrow \text{Sig}(\text{Sign}(sk, m))$ from the issuer;
- generates a succinct proof $\pi_{\text{NIZK}} = \Pi.\text{auxProve}(\text{pk}, x_i, \pi_i)$, where π_i is generated from $\Pi.\text{ZKEval}(\text{pk}, T_{PoP}, (x_{i-1}, \pi_{i-1}))$, where T_{PoP} is an enhanced transformation (defined in Section IV-B2) derived from the transformation T_{PoP} :

$$T_{PoP}(vk, \sigma, m) = V_1(vk, \sigma, m) = (\hat{c}, \hat{\text{aux}});$$

- generates a Sigma proof π_Σ for the relation $\mathcal{R}_{V_2}$ defined in Construction 3 with $(vk, \sigma, \hat{c}, \hat{\text{aux}}; \sigma, \perp, \perp)$;
- outputs a tuple $\text{pt} = (\hat{\sigma}, \hat{c}, \hat{\text{aux}}, \pi_\Sigma, \pi_{\text{NIZK}})$.

To allow rerandomization for the composed proof denoted by $(\sigma, c, \text{aux}, \text{pf}_1, \text{pf}_2)$, we can first apply the transformation T_Σ on the signature σ to obtain a randomized signature $\sigma' = T_\Sigma(\sigma; r_s)$ with randomness r_s . Meanwhile, the malleable NIZK prover applies the transformation as $(c', \text{aux}') = T_{rand}(c, \text{aux}; r_c)$ and uses c' to ensure the implicit link between the newly randomized proof and m .

Definition 12 (Rerandomization). *Consider an already generated presentation tuple $\text{pt} = (\sigma, c, \text{aux}, \pi_\Sigma, \pi_{\text{NIZK}})$, the prover can randomize pt to achieve unlinkability by the following steps by the holder:*

- samples r_c, r_s and randomizes (c, aux, σ) into $(c', \text{aux}', \sigma')$ where $(c', \text{aux}') := T_{rand}(c, \text{aux}, r_c)$ and $\sigma' := T_\Sigma(\sigma, r_s)$;
- generates a succinct proof $\pi'_{\text{NIZK}} \leftarrow \Pi.\text{auxProve}(\text{pk}, x_i, \pi_i)$, where (x_i, π_i) is generated from $\Pi.\text{ZKEval}(\text{pk}, T_{rand}, (x_{i-1}, \pi_{i-1}))$, where T_{rand} is an enhanced transformation derived from the transformation T_{rand} :

$$T_{rand}(c, \text{aux}; r_c) = (c', \text{aux}');$$

- accumulates all randomness along the rerandomization path to $(c', \text{aux}', \sigma')$ to obtain (r_c^*, r_s^*) , generates a Sigma proof π'_Σ for the relation $\mathcal{R}_{V_2}$ with $(vk, \sigma', c', \text{aux}'; \hat{\sigma}, r_c^*, r_s^*)$, and outputs $\text{pt}' = (\sigma', c', \text{aux}', \pi'_\Sigma, \pi'_{\text{NIZK}})$.

Our scheme also supports updating the predicate assertions on attributes, i.e., keep adding new credential attribute predicates $\phi_1, \phi_2, \dots$ to the previous presentation for the same credential and aggregating the new proof with the original one. The definition of the predicate update operation is given below.

Definition 13 (Predicate Update). *To enable selective disclosure of the message m (i.e., proving that m satisfies an attribute predicate $\phi(m) = 1$), the credential prover can update the presentation $\text{pt} = (\sigma, c, \text{aux}, \pi_\Sigma, \pi_{\text{NIZK}})$ to include a new predicate ϕ via the following steps:*

- Generate a succinct proof $\pi'_{\text{NIZK}} \leftarrow \Pi.\text{auxProve}(\text{pk}, x_i, \pi_i)$, where (x_i, π_i) is generated from $\Pi.\text{ZKEval}(\text{pk}, T_{upd_\phi}, (x_{i-1}, \pi_{i-1}))$, where T_{upd_ϕ} is an enhanced transformation derived from the transformation T_{upd_ϕ} :

$$\begin{aligned} T_{upd_\phi}(m, \sigma; r_c) &= (c, \text{aux}, vk), \text{ where} \\ (c, \text{aux}) &= T_c(\hat{c}, \hat{\text{aux}}; r_c) \wedge (\hat{c}, \hat{\text{aux}}) := V_1(vk, \sigma, m), \end{aligned}$$

- Output $\text{pt} = (\sigma, c, \text{aux}, \pi'_\Sigma, \pi'_{\text{NIZK}})$.

Remark 3. Note that the constraint $(\hat{c}, \hat{\text{aux}}) = V_1(vk, \sigma, m) \wedge (c, \text{aux}) = T_c(\hat{c}, \hat{\text{aux}}; r_c)$ can be removed, given that the SNARK is a commit-and-prove version [Campanelli et al.(2019)]. This is because the prover for T_{upd_ϕ} can simply reuse the commitments related to these constraints in the previous step to ensure the binding property. Besides, the result of predicate update is a presentation token $(\sigma, c, \text{aux}, \pi'_1, \pi_2)$. The holder must also apply T_{rand} to rerandomize them for unlinkability of presentations.

B. Protocol Design

In this subsection, we provide a concrete protocol Π_{Re2creds} implementing $\mathcal{F}_{\text{Re2creds}}$ based on the building blocks introduced in Section IV. Π_{Re2creds} interfaces with the ideal functionalities $\mathcal{F}_{ca}$, $\mathcal{F}_{smt}$, and $\mathcal{F}_{crs}$ [Canetti(2001)], [Canetti(2004)]. The required system parameters for signature scheme Sig, Sigma protocol Π_Σ , and the recursive SNARK Π are taken from $\mathcal{F}_{crs}$. We implicitly assume that every protocol participant obtains the issuer's public key from $\mathcal{F}_{ca}$ and system parameters from $\mathcal{F}_{crs}$. The communication between any two parties is implemented by $\mathcal{F}_{smt}$.

1) *Setup*: In the setup phase, the issuer $\mathcal{I}$ generates its verification/secret key pair and registers them with the ideal functionality $\mathcal{F}_{ca}$. The issuer $\mathcal{I}$ runs the following steps on input (SETUP, sid):

1. checks if $\text{sid} = (\mathcal{I}, \text{sid}')$ for some session id sid' ;
2. calculates $(sk, vk) \leftarrow \text{Sig.KeyGen}(1^\lambda)$ and $\pi_{sk} \leftarrow \Pi_\Sigma.\text{Prove}(\text{pp}, vk, sk)^1$ for the relation $\mathcal{R}_{\text{Derive}}$ where $\mathcal{R}_{\text{Derive}}$ implies that vk is correctly derived from sk ;
3. registers the public key (vk, π_{sk}) in $\mathcal{F}_{ca}$;
4. sets attribute value space as $\tilde{\mathbb{A}}$;
5. outputs (SETUPDONE, sid) to $\mathcal{I}$.

2) *Issue*: The issuer $\mathcal{I}$ can issue credentials to any potential credential holder $\mathcal{H}_i$ with allocated attribute values $\vec{a}$. The issuer $\mathcal{I}$ runs the following steps upon receiving input (ISSUE, sid, $\vec{a}_i$, $\mathcal{H}_i$) where $\vec{a}_i \in \tilde{\mathbb{A}}$:

1. proceeds if the caller is the issuer;
2. computes $\sigma \leftarrow \text{Sig.Sign}(sk, \vec{a}_i)$ and sends $(\vec{a}_i, \sigma_i)$ to $\mathcal{H}_i$ over $\mathcal{F}_{smt}$;
3. $\mathcal{H}_i$, upon receiving $(\vec{a}_i, \sigma_i)$ from $\mathcal{I}$ over $\mathcal{F}_{smt}$, checks $\text{Sig.Verify}(vk, \vec{a}_i, \sigma_i) = 1$. It stores $\text{cred} := (\vec{a}_i, \sigma_i)$;
4. outputs (ISSUE, sid, $\vec{a}_i$).

3) *Present*: A credential holder $\mathcal{H}_i$ can present a credential it holds under a session identifier ctx and a set of attribute predicates $\mathcal{T}_\phi$. The $\mathcal{H}_i$ runs the following steps upon receiving input (PRESENT, sid, $\mathcal{T}_\phi$, ctx):

1. retrieves the credential $\text{cred} = (\vec{a}_i, \sigma_i)$, aborts if it does not hold one;
2. determines whether there exists a subset $\mathcal{T}'_\phi \subseteq \mathcal{T}_\phi$, for which presentation proofs have already been generated.
3. If $\mathcal{T}'_\phi = \emptyset$ (no predicate in $\mathcal{T}_\phi$ has been proven before), $\mathcal{H}_i$ generates proofs for $\mathcal{T}_\phi$ from scratch:
 - calculates $(x_1, \pi_1) \leftarrow \Pi.\text{Prove}(\text{pk}, x_0, \pi_0)$;
 - calculates $(\hat{c}, \hat{\text{aux}}) \leftarrow V_1(vk, \sigma_i, \vec{a}_i)$;
 - calculates $(x_2, \pi_2) \leftarrow \Pi.\text{ZKEval}(\text{pk}, \mathcal{T}_{\text{PoP}}, (x_1, \pi_1))$ where x_2 partially construct from $(\hat{c}, \hat{\text{aux}})$;
 - sets $(\sigma, c, \text{aux}) \leftarrow (\sigma_i, \hat{c}, \hat{\text{aux}})$.
- Otherwise, $\mathcal{H}_i$ retrieves a stored state denoted by $(\mathcal{T}'_\phi, \sigma^*, c^*, \text{aux}^*, r_c^*, r_s^*, (x_i, \pi_i))$ and sets $(\sigma, c, \text{aux}, x_2, \pi_2) \leftarrow (\sigma^*, c^*, \text{aux}^*, x_i, \pi_i)$.
4. Next, $\mathcal{H}_i$ sets $\mathcal{T}_\phi^+ := \mathcal{T}_\phi \setminus \mathcal{T}'_\phi$ and runs the following steps:
 - samples r_s, r_c and calculates $(c', \text{aux}') \leftarrow T_{\text{rand}}(c, \text{aux}, r_c)$ and $\sigma' \leftarrow T_\Sigma(\sigma, r_s)$;
 - calculates $(x_3, \pi_3) \leftarrow \Pi.\text{Prove}(\text{pk}, \mathcal{T}_{\text{rand}}, (x_2, \pi_2))$;
 - calculates $\pi_\Sigma \leftarrow \Pi_\Sigma.\text{Prove}(\text{pp}, (vk, \sigma', c', \text{aux}'), (\hat{\sigma}, r_c^*, r_s^*))$ for the relation $\mathcal{R}_{V'_2}$ under ctx , where r_c^*, r_s^* are accumulated randomness from (r_c, r_s) and (r_c^*, r_s^*) ;
 - for $j \leftarrow 1, \dots, |\mathcal{T}_\phi^+|$, calculates $(x_{j+3}, \pi_{j+3}) \leftarrow \Pi.\text{Prove}(\text{pk}, \mathcal{T}_{\text{upd}_{\phi_j}}, x_{j+2}, \pi_{j+2})$ where $\phi_j \in \mathcal{T}_\phi^+$;
 - calculates $\pi_{\text{NIZK}} \leftarrow \Pi.\text{auxProve}(\text{pk}, x_{|\mathcal{T}_\phi^+|+3}, \pi_{|\mathcal{T}_\phi^+|+3})$ under ctx ;
 - stores $(\mathcal{T}_\phi, \sigma', c', \text{aux}', r_c^*, r_s^*, (x_{|\mathcal{T}_\phi^+|+3}, \pi_{|\mathcal{T}_\phi^+|+3}))$ and sets $\text{pt} := (\sigma', c', \text{aux}', \pi_\Sigma, \pi_{\text{NIZK}})$.
5. outputs (TOKEN, sid, pt).

4) *Verify*: A verifier $\mathcal{V}$ can verify the validity of a presentation token against a set of predicates. The verifier $\mathcal{V}$ runs the following steps upon receiving input (Verify, sid, ctx , pt , $\mathcal{T}_\phi$):

1. calculates vk from $\mathcal{T}_\phi$;
2. parses pt into $(\sigma, c, \text{aux}, \pi_\Sigma, \pi_{\text{NIZK}})$;
3. constructs x_i from $\mathcal{T}_\phi$ and (σ, c, aux) ;
4. sets $f_1 \leftarrow \Pi.\text{Verify}(vk, x_i, \pi_{\text{NIZK}})$;
5. sets $f_2 \leftarrow \Pi_\Sigma.\text{Verify}(\text{pp}, (\sigma, c, \text{aux}), \pi_\Sigma)$;
6. outputs (VERIFIED, sid, $f_1 \wedge f_2$).

The above construction satisfies the following theorem. For the detailed proof, please refer to Appendix F.

Theorem 1. *The proposed anonymous credential protocol Π_{Re2creds} securely realizes $\mathcal{F}_{\text{Re2creds}}$, in the $(\mathcal{F}_{crs}, \mathcal{F}_{ca}, \mathcal{F}_{smt})$ -hybrid model, assuming the underlying signature scheme satisfies EUF-CMA, the underlying malleable proof system provides controlled-malleable simulation-extractability and derivation privacy (implied by zero-knowledge), and the Sigma protocol is transformed as a simulation-sound NIZK.*

¹This step is to ensure simulatability of signatures [Chase et al.(2014)].

VI. EVALUATION

TABLE III: Estimated proving constraints of different schemes for various signature algorithms.

Signatures	Re2creds ₁	Re2creds ₂	Re2creds ₃	SP'25	Plonk/Groth16
ECDSA	$l_1 \cdot H + 2 \cdot \mathbb{G}^+ + \phi \approx$ $l_1 \cdot 2^{13} + 2 \cdot 2^9 + \phi $	$1 \cdot \mathbb{G}^+ + \phi \approx$ $2^9 + \phi $	$1 \cdot \mathbb{G}^+ \approx 2^9$	$l_1 \cdot H + 1 \cdot \mathbb{G}^\times + \phi \approx$ $l_1 \cdot 2^{13} + 2^{16} + \phi $	$l_1 \cdot H + 2 \cdot \mathbb{G}^\times + \phi \approx$ $l_1 \cdot 2^{15} + 2 \cdot 2^{20} + \phi $
RSA-PSS	$l_1 \cdot H + 1 \cdot p + 1 \cdot e + \phi \approx$ $l_1 \cdot 2^{13} + 2^{16} + 2^{14} + \phi $	$\phi \approx \phi $	-	$l_1 \cdot H + 1 \cdot \mathbb{G}^\times + \phi \approx$ $l_1 \cdot 2^{13} + 2^{16} + \phi $	$l_1 \cdot H + 2 \cdot \mathbb{G}^\times + \phi \approx$ $l_1 \cdot 2^{15} + 2 \cdot 2^{20} + \phi $
BLS	$l_1 \cdot H + 2 \cdot \mathbb{G}_1^+ + \phi \approx$ $l_1 \cdot 2^{13} + 2 \cdot 2^9 + \phi $	$1 \cdot \mathbb{G}_1^+ + \phi \approx$ $2^9 + \phi $	$1 \cdot \mathbb{G}_1^+ \approx 2^9$	-	$l_1 \cdot H + 2P + \phi \approx$ $l_1 \cdot 2^{15} + 2 \cdot 2^{22} + \phi $

¹ l_1 represents the length of signed message (in 512-bit chunks) and H denotes the hash operation over a 512-bit chunk.

² ϕ denotes the predicate check in circuit with $|\phi|$ constraints. e denotes big integer exponentiation, and p denotes PSS padding algorithm.

³ P denotes the pairing operation. $\mathbb{G}_1^+, \mathbb{G}^+$ denote point addition operations. $\mathbb{G}^\times$ denotes point exponentiation operations.

⁴ Plonk/Groth16 refers to directly proving signature verification without optimizations such as lookup tables.

⁵ SP'25 does not support BLS signatures, so it is marked with a dash. Re2creds₃ for RSA-PSS can be implemented via a dummy randomization circuit.

TABLE IV: Estimated proof sizes and verification costs of different schemes for various signature algorithms.

Scheme	Signatures	Presentation Size	Verification Cost
Re2creds	ECDSA	$13 \mathbb{G}_1 + 9 \mathbb{F} + 2 \mathbb{G}' + 3 \mathbb{F}' $	$2P + 22\mathbb{G}_1 + 4\mathbb{G}'$
Re2creds	RSA-PSS	$13 \mathbb{G}_1 + 9 \mathbb{F} $	$2P + 22\mathbb{G}_1$
Re2creds	BLS	$13 \mathbb{G}_1 + 9 \mathbb{F} + 2\mathbb{G}'_t + 2\mathbb{F}' + 2\mathbb{G}'_1$	$2P + 22\mathbb{G}_1 + 3\mathbb{G}'_t + 2P'$
SP'25	ECDSA	$3 \mathbb{G}_1 + 1 \mathbb{G}_2 + 4 \mathbb{G}' + 4 \mathbb{F}' $	$4P + l\mathbb{G}_1 + 10\mathbb{G}'$
SP'25	RSA-PSS	$3 \mathbb{G}_1 + 1 \mathbb{G}_2 $	$4P + l\mathbb{G}_1$
Plonk	ECDSA,BLS,RSA-PSS	$7 \mathbb{G}_1 + 7 \mathbb{F} $	$2P + 18\mathbb{G}_1$
Groth16	ECDSA,BLS,RSA-PSS	$2 \mathbb{G}_1 + 1 \mathbb{G}_2 $	$3P + l\mathbb{G}_1$

¹ $|\cdot|$ means the length of an element in a group or field and l denotes the number of public inputs.

² $P, \mathbb{G}_1, \mathbb{G}_2$ denote the proof system's pairing operation, group point exponentiation operations, respectively. Signatures' underlying curves' operations or elements are marked with a superscript like $\mathbb{G}'_t, \mathbb{G}', P', \mathbb{F}'$.

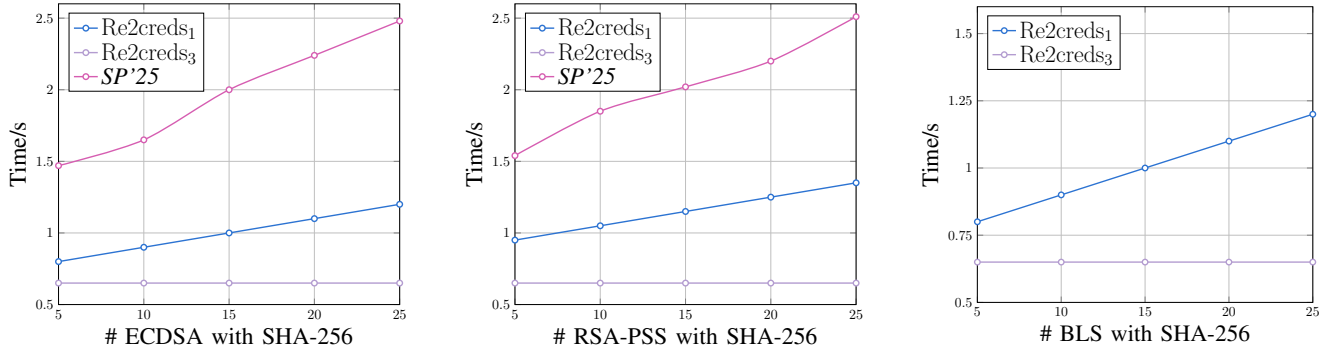


Fig. 3: The evaluation results for presentation costs of different signature schemes with varying-size messages. The x -axis denotes the size of signed message in a credential, which is the multiple of 512 bits.

In this section, we present the prototype implementation of Re2creds and evaluate its performance.

A. Setup and Implementation

1) *Experimental Environment:* All benchmarks were conducted on a Mac Air equipped with an M1 CPU, 16 GB of RAM, and a 256 GB SSD. This configuration represents a typical consumer-grade device with moderate computational capabilities.

2) *Implementation:* Our system is implemented in Rust (v1.90.0-nightly). We set the security parameter to $\lambda = 128$ and utilize the Scroll's fork of Halo2, instantiated with the KZG polynomial commitment scheme. The proof system operates over the BN254/Grumpkin cycle of curves. We evaluate three signature schemes: BLS over the BN254 curve, ECDSA over P-256, and RSA-PSS. SHA-256 is used as the hash function throughout.

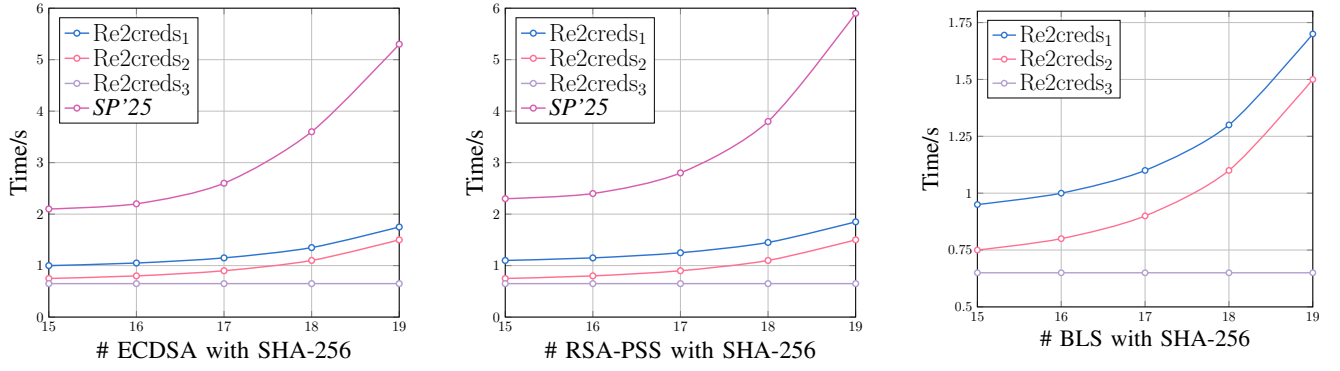


Fig. 4: The evaluation results for presentation costs of different signature schemes with varying-size predicates. The x -axis denotes the size of proving predicates, which is measured by constraints in $\log_2$ scaling.

3) *Baselines*: We select the state-of-the-art Proof of Possession (PoP) scheme for legacy signatures, SP'25 [Woo et al.(2025)], as our baseline. SP'25 optimizes proof generation via lookup tables. To enable a fair comparison, we adapt SP'25 into an anonymous credential system by integrating predicate checks within its circuits. Besides, we also consider generic zk-SNARKs (Plonk/Groth16) as additional baselines, which directly prove signature verification and predicate checks without optimizations. To demonstrate the efficiency of Re2creds, we evaluate three variants:

- Re2creds₁: Generates a fresh presentation from scratch for each request without reusing prior proofs.
- Re2creds₂: Derives a new presentation from an existing malleable PoP (Section IV-B) using predicate updates and rerandomization.
- Re2creds₃: Derives a new presentation directly from an existing presentation via the rerandomization mechanism.

B. Theoretical Analysis

We first conduct a theoretical analysis of the computational and communication overhead of different schemes. The detailed comparison results are presented in Table III and Table IV.

Proving cost: Table III illustrates the estimated circuit constraints, which serve as a primary metric for proving complexity. For fresh credential generation (Re2creds₁), our scheme achieves performance comparable to SP'25, where the dominant cost stems from hashing the message and the predicate check. However, for the more frequent operations of predicate updates (Re2creds₂) and presentation rerandomization (Re2creds₃), Re2creds demonstrates superior efficiency. Specifically, Re2creds₂ reduces the proving cost to near-constant size (approx. 2^9 constraints plus predicate logic $|\phi|$), as it allows updating the credential without re-evaluating the costly signature verification (which involves heavy operations like pairings or large exponentiations). For example, with RSA-PSS, the update cost is determined solely by $|\phi|$ predicate constraints. Re2creds₃ further minimizes the cost to a negligible level ($\approx 2^9$) by bypassing predicate re-evaluation entirely. In contrast, SP'25 and generic zk-SNARKs require full re-proving of the signature and predicates for every presentation, resulting in consistently high overheads (e.g., $> 2^{20}$ constraints for generic methods involving pairings).

Communication and verification: Table IV reports the proof sizes and verification costs. Re2creds produces compact presentations (e.g., ≈ 13 group elements for ECDSA/BLS), which are slightly larger than SP'25 but remain efficient for transmission. Crucially, our verification cost is constant ($O(1)$) with respect to the message length, requiring only a fixed number of pairings (e.g., 2 for ECDSA/RSA) and scalar multiplications. This outperforms SP'25, where verification complexity grows linearly with the message size (i.e., l_1). Although Groth16 yields smaller proofs, its prohibitive proving cost renders it unsuitable for efficient client-side privacy-preserving applications. Overall, Re2creds achieves an optimal trade-off between prover efficiency and verifier scalability, enabling lightweight and scalable credential management.

C. Experimental Results

Credential presentation time is primarily influenced by two factors: the length of the signed messages (attributes) and the complexity of the predicates. To align with the SHA-256 chunk size, we set attribute lengths as multiples of 512 bits. We evaluate efficiency by varying the predicate size from 2^{15} to 2^{19} constraints. As SP'25 does not support BLS signatures, it is excluded from the BLS evaluations. Results are summarized in Figure 3 and Figure 4. Here, we disregard naive AC schemes that directly use zk-SNARK (i.e., Plonk and Groth16) to prove signature verification without optimizations, as they are known to be inefficient, which can result in tens of seconds to generate a PoP without predicate checks.

Impact of message size: Figure 3 illustrates credential presentation efficiency as the signed message size varies from 5×512 to 25×512 bits. This experiment focuses on core proving costs without additional predicates. Results show that presentation

time for both the baseline SP'25 and Re2creds₁ grows linearly with message size, primarily due to increased constraints for hashing larger messages. However, Re2creds₁ consistently outperforms SP'25 with a significantly lower growth rate. For example, with a 25×512 -bit ECDSA message, Re2creds₁ reduces generation time by approximately 50% compared to SP'25 (1.2s vs. 2.48s). Furthermore, Re2creds₃ achieves constant-time performance (≈ 0.65 s), leveraging lightweight rerandomization independent of attribute length.

Impact of predicate size: Figure 4 evaluates the impact of predicate complexity (2^{15} to 2^{19} constraints) with a fixed message size of 10×512 bits. SP'25 shows a sharp increase in cost as predicate size grows, exceeding 5.3s for the most complex predicate. In contrast, our system maintains superior efficiency. Re2creds₁ significantly reduces overhead, while Re2creds₂ further lowers latency (e.g., 1.5s for the largest predicate) by updating proofs via the malleable PoP structure. Notably, Re2creds₃ exhibits a constant cost of about 0.65s, unaffected by predicate size, demonstrating the efficiency of our rerandomization technique.

Verification cost: Both Re2creds and SP'25 utilize zk-SNARKs for credential generation, ensuring that the proof size and verification cost for the circuit component remain fixed. For the Sigma protocol components executed outside the circuit, their simple verification relations result in constant and negligible proof sizes and verification costs. Overall, the proof sizes for all schemes do not exceed 2 KB, and the verification times are less than 50 ms. These minor differences are negligible in practical deployment.

VII. RELATED WORK

We summarize related works on anonymous credentials and privacy-preserving identity systems, as well as recursive argument systems.

Anonymous credentials: Camenisch and Lysyanskaya [Camenisch and Lysyanskaya(2001)] proposed the first fully anonymous credential scheme using zero-knowledge proofs to prove possession of a signature on a pseudonym. Similarly, Belenkiy [Belenkiy et al.(2008)] introduced P-signatures using commitments and signatures. Zhang et al. [Zhang and Feng(2012)] and Sonobe et al. [Sonnino et al.(2018)] (Coconut) extended these concepts to attribute-based signatures and distributed scenarios, respectively. Verheul [Verheul(2001)] and later Hanser and Slamanig [Fuchsbaauer et al.(2019)] explored randomizable credentials, with the latter introducing SPS-EQ for selective disclosure. Subsequent works have leveraged SPS-EQ for delegatable [Connolly et al.(2022)] and threshold [Mir et al.(2023b)] credentials. Other constructions address specific constraints such as resource-limited devices [Hanzlik and Slamanig(2021)], updatability [Blömer et al.(2019)], and issuer-hiding [Mir et al.(2023a)].

Privacy-preserving identity systems: Several schemes build privacy-preserving identities using cryptographic proofs. UCBlocker [Du et al.] uses verifiable credentials and Sigma protocols for access control. CanDID [Maram et al.(2021)] migrates legacy identities to a decentralized system managed by a committee, providing inter-application anonymity but lacking intra-application unlinkability. Recently, zk-creds [Rosenberg et al.(2023)] and Zebra [Rathee et al.(2022)] employ zk-SNARKs to prove possession of credentials and attribute satisfaction for web and on-chain access control, respectively.

Recursive argument systems: Recursive SNARKs/IOPs allow proving the validity of computations involving other proofs, enabling succinct verification of arbitrary depth. Halo [Bowe et al.(2019)] introduced an accumulation scheme for IVC and PCD [Valiant(2008)], [Bünz et al.(2021)] that defers expensive verification steps to the end, significantly reducing overhead. Nova improves on this via a folding scheme for NP instances, reducing cryptographic costs but potentially increasing proof generation time. SuperNova [Kothapalli and Setty(2022)] extends this to multiple predicates, though challenges remain regarding efficiency in complex recursive scenarios involving multiple inputs or zero-knowledge requirements.

VIII. CONCLUSION

In this paper, we introduced Re2creds, a novel anonymous credential system designed to address critical limitations in efficiency and compatibility that hinder the widespread adoption of decentralized identity. Our system pioneers a reusable credential presentation paradigm, enabled by a new malleable NIZK construction called Malo, which significantly reduces computational overhead by allowing presentations to be efficiently updated and rerandomized. Furthermore, we developed a proof combination mechanism that seamlessly integrates widely-deployed legacy signatures, overcoming a major barrier to real-world adoption. By enhancing both usability and compatibility, Re2creds provides a practical and scalable foundation for building secure, private, and user-centric identity solutions in the Web3 ecosystem.

REFERENCES

- [Attema and Cramer(2020)] Thomas Attema and Ronald Cramer. 2020. Compressed-protocol theory and practical application to plug & play secure algorithmics. In *Annual International Cryptology Conference*. Springer, 513–543.
- [Belenkiy et al.(2008)] Mira Belenkiy, Melissa Chase, Markulf Kohlweiss, and Anna Lysyanskaya. 2008. P-signatures and noninteractive anonymous credentials. In *Theory of Cryptography: Fifth Theory of Cryptography Conference, TCC 2008, New York, USA, March 19-21, 2008. Proceedings 5*. Springer, 356–374.
- [Ben-Sasson et al.(2018)] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. 2018. Fast reed-solomon interactive oracle proofs of proximity. In *45th international colloquium on automata, languages, and programming (icalp 2018)*. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, 14–1.

- [Blömer et al.(2019)] Johannes Blömer, Jan Bobolz, Denis Diemert, and Fabian Eidens. 2019. Updatable anonymous credentials and applications to incentive systems. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*. 1671–1685.
- [Boneh et al.(2021)] Dan Boneh, Justin Drake, Ben Fisch, and Ariel Gabizon. 2021. Halo infinite: Proof-carrying data from additive polynomial commitments. In *Advances in Cryptology—CRYPTO 2021: 41st Annual International Cryptology Conference, CRYPTO 2021, Virtual Event, August 16–20, 2021, Proceedings, Part I 41*. Springer, 649–680.
- [Boneh et al.(2019)] Dan Boneh, Sergey Gorbunov, Hoeteck Wee, and Zhenfei Zhang. 2019. BLS signature scheme. *Tech. Rep.* (2019).
- [Bowe et al.(2019)] Sean Bowe, Jack Grigg, and Daira Hopwood. 2019. Recursive proof composition without a trusted setup. *Cryptology ePrint Archive* (2019).
- [Bünz et al.(2018)] Benedikt Bünz, Jonathan Bootle, Dan Boneh, Andrew Poelstra, Pieter Wuille, and Greg Maxwell. 2018. Bulletproofs: Short proofs for confidential transactions and more. In *2018 IEEE symposium on security and privacy (SP)*. IEEE, 315–334.
- [Bünz et al.(2021)] Benedikt Bünz, Alessandro Chiesa, William Lin, Pratyush Mishra, and Nicholas Spooner. 2021. Proof-carrying data without succinct arguments. In *Annual International Cryptology Conference*. Springer, 681–710.
- [Camenisch and Lysyanskaya(2001)] Jan Camenisch and Anna Lysyanskaya. 2001. An efficient system for non-transferable anonymous credentials with optional anonymity revocation. In *Advances in Cryptology—EUROCRYPT 2001: International Conference on the Theory and Application of Cryptographic Techniques Innsbruck, Austria, May 6–10, 2001 Proceedings 20*. Springer, 93–118.
- [Campanelli et al.(2019)] Matteo Campanelli, Dario Fiore, and Anaïs Querol. 2019. LegoSNARK: Modular design and composition of succinct zero-knowledge proofs. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*. 2075–2092.
- [Canetti(2000)] Ran Canetti. 2000. Universally Composable Security: A New Paradigm for Cryptographic Protocols. *Cryptology ePrint Archive* (2000).
- [Canetti(2001)] Ran Canetti. 2001. Universally composable security: A new paradigm for cryptographic protocols. In *Proceedings 42nd IEEE Symposium on Foundations of Computer Science*. IEEE, 136–145.
- [Canetti(2004)] Ran Canetti. 2004. Universally composable signature, certification, and authentication. In *Proceedings. 17th IEEE Computer Security Foundations Workshop, 2004*. IEEE, 219–233.
- [Chakraborty et al.(2025)] Suvradip Chakraborty, Dennis Hofheinz, Roman Langrehr, Jesper Buus Nielsen, Christoph Striecks, and Daniele Venturi. 2025. Malleable SNARKs and Their Applications. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 184–213.
- [Chase et al.(2012)] Melissa Chase, Markulf Kohlweiss, Anna Lysyanskaya, and Sarah Meiklejohn. 2012. Malleable proof systems and applications. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 281–300.
- [Chase et al.(2014)] Melissa Chase, Markulf Kohlweiss, Anna Lysyanskaya, and Sarah Meiklejohn. 2014. Malleable signatures: New definitions and delegatable anonymous credentials. In *2014 IEEE 27th computer security foundations symposium*. IEEE, 199–213.
- [Chiesa et al.(2020)] Alessandro Chiesa, Yuncong Hu, Mary Maller, Pratyush Mishra, Noah Vesely, and Nicholas Ward. 2020. Marlin: Preprocessing zkSNARKs with universal and updatable SRS. In *Advances in Cryptology—EUROCRYPT 2020: 39th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zagreb, Croatia, May 10–14, 2020, Proceedings, Part I 39*. Springer, 738–768.
- [Connolly et al.(2022)] Aisling Connolly, Pascal Lafourcade, and Octavio Perez Kempner. 2022. Improved constructions of anonymous credentials from structure-preserving signatures on equivalence classes. In *IACR International Conference on Public-Key Cryptography*. Springer, 409–438.
- [Damgård and Koprowski(2001)] Ivan Damgård and Maciej Koprowski. 2001. Practical Threshold RSA Signatures without a Trusted Dealer. In *Advances in Cryptology — EUROCRYPT 2001*, Birgit Pfizmann (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 152–165.
- [Du et al.(2023)] Changlai Du, Hexuan Yu, Yang Xiao, Y Thomas Hou, Angelos D Keromytis, and Wenjing Lou. 2023. UCBlocker: Unwanted call blocking using anonymous authentication. In *32nd USENIX Security Symposium (USENIX Security 23)*. 445–462.
- [Explorations([n. d.])] Privacy & Scaling Explorations. [n. d.]. halo2curves. <https://github.com/privacy-scaling-explorations/halo2curves.git/>.
- [Fuchsbauer et al.(2019)] Georg Fuchsbauer, Christian Hanser, and Daniel Slamanig. 2019. Structure-preserving signatures on equivalence classes and constant-size anonymous credentials. *Journal of Cryptology* 32 (2019), 498–546.
- [Gabizon et al.(2019)] Ariel Gabizon, Zachary J Williamson, and Oana Ciobotaru. 2019. Plonk: Permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge. *Cryptology ePrint Archive* (2019).
- [Grassi et al.(2021)] Lorenzo Grassi, Dmitry Khovratovich, Christian Rechberger, Arnab Roy, and Markus Schofnegger. 2021. Poseidon: A new hash function for {Zero-Knowledge} proof systems. In *30th USENIX Security Symposium (USENIX Security 21)*. 519–535.
- [Groth(2006)] Jens Groth. 2006. Simulation-sound NIZK proofs for a practical language and constant size group signatures. In *International Conference on the Theory and Application of Cryptology and Information Security*. Springer, 444–459.
- [Groth(2016)] Jens Groth. 2016. On the size of pairing-based non-interactive arguments. In *Advances in Cryptology—EUROCRYPT 2016: 35th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Vienna, Austria, May 8–12, 2016, Proceedings, Part II 35*. Springer, 305–326.
- [Hanzlik and Slamanig(2021)] Lucjan Hanzlik and Daniel Slamanig. 2021. With a little help from my friends: Constructing practical anonymous credentials. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*. 2004–2023.
- [Kadianakis et al.(2023)] George Kadianakis, Mary Maller, and Andrija Novakovic. 2023. Sigmabus: binding sigmas in circuits for fast curve operations. *Cryptology ePrint Archive* (2023).
- [Kate et al.(2010)] Aniket Kate, Gregory M Zaverucha, and Ian Goldberg. 2010. Constant-size commitments to polynomials and their applications. In *International conference on the theory and application of cryptography and information security*. Springer, 177–194.
- [Kothapalli and Setty(2022)] Abhiram Kothapalli and Srinath Setty. 2022. SuperNova: Proving universal machine executions without universal circuits. *Cryptology ePrint Archive* (2022).
- [Kothapalli et al.(2022)] Abhiram Kothapalli, Srinath Setty, and Ioanna Tzialla. 2022. Nova: Recursive zero-knowledge arguments from folding schemes. In *Annual International Cryptology Conference*. Springer, 359–388.
- [Libert(2024)] Benoît Libert. 2024. Simulation-extractable KZG polynomial commitments and applications to HyperPlonk. In *IACR International Conference on Public-Key Cryptography*. Springer, 68–98.
- [Maram et al.(2021)] Deepak Maram, Harjasleen Malvai, Fan Zhang, Nerla Jean-Louis, Alexander Frolov, Tyler Kell, Tyrone Lobban, Christine Moy, Ari Juels, and Andrew Miller. 2021. Candid: Can-do decentralized identity with legacy compatibility, sybil-resistance, and accountability. In *2021 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1348–1366.
- [Mir et al.(2023a)] Omid Mir, Balthazar Bauer, Scott Griffy, Anna Lysyanskaya, and Daniel Slamanig. 2023a. Aggregate Signatures with Versatile Randomization and Issuer-Hiding Multi-Authority Anonymous Credentials. *Cryptology ePrint Archive* (2023).
- [Mir et al.(2023b)] Omid Mir, Daniel Slamanig, and René Mayrhofer. 2023b. Threshold Delegatable Anonymous Credentials with Controlled and Fine-Grained Delegation. *IEEE Transactions on Dependable and Secure Computing* (2023).
- [Naveh and Tromer(2016)] Assa Naveh and Eran Tromer. 2016. Photoproof: Cryptographic image authentication for any set of permissible transformations. In *2016 IEEE Symposium on Security and Privacy (SP)*. IEEE, 255–271.
- [Nguyen et al.(1999)] Khanh Quoc Nguyen, Feng Bao, Yi Mu, and Vijay Varadharajan. 1999. Zero-knowledge proofs of possession of digital signatures and its applications. In *International Conference on Information and Communications Security*. Springer, 103–118.

- [Rathee et al.(2022)] Deevashwer Rathee, Guru Vamsi Policharla, Tiancheng Xie, Ryan Cottone, and Dawn Song. 2022. ZEBRA: SNARK-based Anonymous Credentials for Practical, Private and Accountable On-chain Access Control. *Cryptology ePrint Archive*, Paper 2022/1286. <https://eprint.iacr.org/2022/1286> <https://eprint.iacr.org/2022/1286>
- [Rosenberg et al.(2022)] Michael Rosenberg, Mary Maller, and Ian Miers. 2022. SNARKBlock: Federated anonymous blocklisting from hidden common input aggregate proofs. In *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 948–965.
- [Rosenberg et al.(2023)] Michael Rosenberg, Jacob White, Christina Garman, and Ian Miers. 2023. zk-creds: Flexible anonymous credentials from zksnarks and existing identity infrastructure. In *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE, 790–808.
- [Schnorr(1991)] Claus-Peter Schnorr. 1991. Efficient signature generation by smart cards. *Journal of cryptology* 4, 3 (1991), 161–174.
- [Shi et al.(2023)] Rui Shi, Huamin Feng, Yang Yang, Feng Yuan, Yingjiu Li, Hwee Hwa Pang, and Robert H. Deng. 2023. Threshold Attribute-Based Credentials With Redactable Signature. *IEEE Transactions on Services Computing* 16, 5 (2023), 3751–3765.
- [Song(2023)] Rui Song. 2023. LinkDID: A Privacy-Preserving, Sybil-Resistant and Key-Recoverable Decentralized Identity Scheme. *arXiv preprint arXiv:2307.14679* (2023).
- [Sonnino et al.(2018)] Alberto Sonnino, Mustafa Al-Bassam, Shehar Bano, Sarah Meiklejohn, and George Danezis. 2018. Coconut: Threshold Issuance Selective Disclosure Credentials with Applications to Distributed Ledgers. *arXiv e-prints*, Article arXiv:1802.07344 (Feb. 2018), arXiv:1802.07344 pages. <https://doi.org/10.48550/arXiv.1802.07344> arXiv:1802.07344 [cs.CR]
- [Valiant(2008)] Paul Valiant. 2008. Incrementally verifiable computation or proofs of knowledge imply time/space efficiency. In *Theory of Cryptography: Fifth Theory of Cryptography Conference, TCC 2008, New York, USA, March 19-21, 2008. Proceedings* 5. Springer, 1–18. https://doi.org/10.1007/978-3-540-78524-8_1
- [Vanstone(1992)] Scott Vanstone. 1992. Responses to NIST’s proposal. *Commun. ACM* 35, 7 (1992), 50–52.
- [Verheul(2001)] Eric R Verheul. 2001. Self-blindable credential certificates from the Weil pairing. In *International Conference on the Theory and Application of Cryptology and Information Security*. Springer, 533–551.
- [Wang et al.(2023)] Ke Wang, Jianbo Gao, Qiao Wang, Jiashuo Zhang, Yue Li, Zhi Guan, and Zhong Chen. 2023. Hades: Practical decentralized identity with full accountability and fine-grained sybil-resistance. In *Proceedings of the 39th Annual Computer Security Applications Conference*. 216–228.
- [Woo et al.(2025)] Anna PY Woo, Alex Ozdemir, Chad Sharp, Thomas Pornin, and Paul Grubbs. 2025. Efficient Proofs of Possession for Legacy Signatures. In *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE, 3291–3308.
- [Xie et al.(2024)] Sijiang Xie, Rui Shi, Yang Yang, Huiqin Xie, Yingjiu Li, and Robert H Deng. 2024. AccCred: Improved Accountable Anonymous Credentials With Dynamic Triple-Hiding Committees. *IEEE Transactions on Dependable and Secure Computing* (2024).
- [Xue et al.(2025)] Yuxi Xue, Tianyu Zheng, Shang Gao, Bin Xiao, and Man Ho Au. 2025. Compressed sigma protocols: new model and aggregation techniques. In *Australasian Conference on Information Security and Privacy*. Springer, 163–183.
- [Zhang and Feng(2012)] Yan Zhang and Dengguo Feng. 2012. Efficient attribute proofs in anonymous credential using attribute-based cryptography. In *International Conference on Information and Communications Security*. Springer, 408–415.
- [Zheng et al.(2023)] Tianyu Zheng, Shang Gao, Yu Guo, and Bin Xiao. 2023. Kilonova: Non-uniform PCD with zero-knowledge property from generic folding schemes. *Cryptology ePrint Archive* (2023).

APPENDIX

A. Definitions for Digital Signature

Definition 14 (EUF-CMA). A digital signature scheme is EUF-CMA secure if and only if for any PPT adversary $\mathcal{A}$, there exists a negligible function negl such that $\Pr \left[\text{Exp}_{\mathcal{A}}^{\text{EUF-CMA}}(\lambda) = 1 \right] \leq \text{negl}(\lambda)$, where the experiment $\text{Exp}_{\mathcal{A}}^{\text{EUF-CMA}}(\lambda)$ is defined in Fig. 5.

$\text{Exp}_{\mathcal{A}}^{\text{EUF-CMA}}(\lambda)$:
1 : $\text{pp} \leftarrow \Sigma.\text{Setup}(1^\lambda)$
2 : $(sk, pk) \leftarrow \Sigma.\text{KeyGen}(1^\lambda)$
3 : $\mathcal{Q} \leftarrow \emptyset, (m^*, \sigma^*) \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Sign}}}(pk)$
4 : return 1 if $\Sigma.\text{Verify}(pk, m^*, \sigma^*) = 1 \wedge m^* \notin \mathcal{Q}$
5 : return 0
$\mathcal{O}_{\text{Sign}}(m)$:
$\sigma \leftarrow \Sigma.\text{Sign}(sk, m), \mathcal{Q} \leftarrow \mathcal{Q} \cup \{m\}$
1 : return σ

Fig. 5: The EUF-CMA experiment for Σ .

B. Definitions for KZG

To ensure *Simulation Extractability* of the constructed argument systems, we require the KZG scheme to satisfy the following definition according to [Libert(2024)].

Definition 15 (Simulation Extractability (informal)). A KZG PCS is simulation-extractable if, for any PPT adversary $\mathcal{A}$ with an oracle access to a PPT simulator $\mathcal{S}$ that returns a simulated proof for an input statement, There exists a PPT extractor $\mathcal{E}_{\mathcal{A}}$

that has access to $\mathcal{A}$'s internal state/randomness, which can extract a valid witness f for a statement x . Besides, the probability is negligible: f is not a satisfying witness for x , meanwhile x is not queried to $\mathcal{S}$.

We present the KZG aggregation scheme in Figure 6. Given k independent relations $\mathcal{R}_i = \{(f_i; C_i, z_i, v_i) : C_i = \text{KZG.Commit}(\text{pk}, f_i) \wedge f_i(z_i) = v_i\}$, the prover and verifier run the following protocol to aggregate them into one relation $\mathcal{R}^*$.

Lemma 2. *If relation $\mathcal{R}^*$ is valid, given that the polynomial commitment scheme KZG satisfies the completeness and extractability, the protocol in Figure 6 is a proof of knowledge for the aggregation relation $\mathcal{R}_{agg}(\text{pk}, \text{vk}, d) :=$*

$$\left\{ (\vec{f}; (\vec{C}, \vec{z}, \vec{v})) : \begin{array}{l} \vec{f} = (f_1, \dots, f_k) \text{ s.t. } f_i \in \mathbb{F}_p^{(<d)}[X] \\ \forall i \in [k] \ f_i(z_i) = v_i \\ \forall i \in [k] \ C_i = \text{KZG.Commit}(\text{pk}, f_i) \end{array} \right\}$$

The security proof of Lemma 2 can be referred to [Boneh et al.(2021)].

KZG Aggregation Scheme

- 1 : $\mathcal{P}$'s inputs: $(\text{pk}, (f_1, z_1), \dots, (f_k, z_k))$
- 2 : $\mathcal{V}$'s inputs: $(\text{vk}, (C_1, z_1, v_1), \dots, (C_k, z_k, v_k))$
- 3 : $\mathcal{P}, \mathcal{V} : z(X) := \prod_{i=1}^k (X - z_i), z_i(X) := z(X)/(X - z_i)$
- 4 : $\mathcal{V} \rightarrow \mathcal{P} : \rho \xleftarrow{\$} \mathbb{F}_p$
- 5 : $\mathcal{P} : q(X) = \sum_{i=1}^k \rho^{i-1} \cdot (f_i(X) - f_i(z_i)) \cdot \frac{z_i(X)}{z(X)}$.
- 6 : $C_q \leftarrow \text{KZG.Commit}(\text{pk}, q)$
- 7 : $\mathcal{P} \rightarrow \mathcal{V} : C_q$
- 8 : $\mathcal{V} \rightarrow \mathcal{P} : r \xleftarrow{\$} \mathbb{F}_p$
- 9 : $\mathcal{P}$ computes $f^*(X) := :$
- 10 : $\sum_{i=1}^k \rho^{i-1} \cdot z_i(r) \cdot (f_i(X) - f_i(z_i)) - z(r) \cdot q(X)$
- 11 : $\mathcal{V} : C^* = \sum_{i=1}^k \rho^{i-1} \cdot z_i(r) \cdot (C_i - v_i \cdot G) - z(r) \cdot C_q$
- 12 : $\mathcal{V} : v^* = \sum_{i=1}^k \rho^{i-1} \cdot v_i$
- 13 : $\mathcal{P}$ and $\mathcal{V}$ aggregate to $(f^*; C^*, r, v^*) \in \mathcal{R}^*$

Fig. 6: KZG Aggregation Scheme

Note that $\mathcal{R}^*$ is defined as

$$\mathcal{R}^* = \left\{ (f^*; C^*, r, v^*) : \begin{array}{l} C^* = \text{KZG.Commit}(\text{pk}, f^*) \wedge f^*(r) = v^* \end{array} \right\}$$

C. Definitions for Malleable NIZK

For *soundness*, the malleable arguments should satisfy controlled-malleable simulation extractability as defined in [Chakraborty et al.(2025)] and it is briefly sketched as follows.

Definition 16 (Controlled-malleable Simulation Intractability). *A malleable non-interactive zero-knowledge argument system $\Pi = (\text{Setup}, \text{Prove}, \text{Verify})$ for an NP relation $\mathcal{R}$ and an admissible set of transformations $\mathcal{T}$ is controlled-malleable simulation extractability if for every PPT adversary $\mathcal{A}$ there exists a PPT extractor $\mathcal{E}$ such that*

$$\text{Adv}_{\mathcal{A}, \mathcal{E}, \Pi}^{\text{cm-sse}}(\lambda) = |\Pr[\text{Exp}_{\mathcal{A}, \mathcal{E}, \Pi}^{\text{cm-sse}}(\lambda) = 1]| \leq \text{negl}(\lambda),$$

where the experiment is defined in Figure 8.

```

 $\text{Exp}_{\mathcal{A}, \Pi = (\text{Setup}, \text{Prove}, \text{Verify})}^{\text{rdp}}(\lambda) :$ 
1 :  $b \leftarrow \{0, 1\}$ 
2 :  $(\text{pp}, \text{td}) \leftarrow S_1(1^\lambda)$ 
3 :  $(\text{st}, (x_i, \pi_i)_{i=1}^n, T = (T_x, T_w)) \leftarrow \mathcal{A}_1(\text{pp})$ 
4 : if  $\exists i \in [n] : \text{Verify}(\text{pp}, x_i, \pi_i) = 0$  then return  $b' \leftarrow \{0, 1\}$ 
5 : if  $b = 0$  then  $\pi \leftarrow S_2(\text{pp}, \text{td}, T_x(x_1, \dots, x_n))$ 
6 :     else  $\pi \leftarrow \text{ZKEval}(\text{pp}, T, (x_i, \pi_i)_{i=1}^n)$ 
7 :  $b' \leftarrow \mathcal{A}_2(\text{st}, \pi)$ 
8 : return  $b \stackrel{?}{=} b'$ 

```

Fig. 7: The derivation privacy experiment.

```

 $\text{Exp}_{\mathcal{A}, \mathcal{E}, \Pi = (\text{Setup}, \text{Prove}, \text{Verify})}^{\text{cm-sse}}(\lambda) :$ 
1 :  $Q_{\text{sim}} := \emptyset.$ 
2 :  $(\text{pp}, \text{td}) \leftarrow S_1(1^\lambda)$ 
3 :  $(x, \pi) \leftarrow \mathcal{A}^{\text{PROVE}(\cdot)}(1^\lambda, \text{pp} : r)$ 
4 :  $E \leftarrow \mathcal{E}(1^\lambda, \text{pp}, Q_{\text{sim}}, r)$ 
5 : if  $\text{checkExp}(x, E) = 1$  then return 0, else 1

 $\text{PROVE}(x) :$ 
1 :  $\pi \leftarrow S_2(\text{pp}, \text{td}, x)$ 
2 :  $Q_{\text{sim}} := Q_{\text{sim}} \cup \{(x, \pi)\}$ 
3 : return  $\pi$ 

 $\text{checkExp}(x, E) :$ 
if  $(x, \cdot) \in Q_{\text{sim}}$  then return 1
1 : if  $\exists w : (x, w) \in E \wedge (x, w) \in \mathcal{R}$  then return 1
2 : if  $\exists T = (T_x, T_w), (x_i)_{i=1}^n : (x, (T, (x_i)_{i=1}^n)) \in E$  then:
3 :     if  $T \in \mathcal{T} \bigwedge x = T_x(x_1, \dots, x_n)$  then let  $E' := E \setminus \{(x, w) \in E\}$ , return  $\bigwedge_{i=1}^n \text{checkExp}(x_i, E')$ 
4 : else return 0

```

Fig. 8: The simulation intractability experiment for a malleable argument Π

D. NIZK

Definition 17 (Perfect Completeness). *An NIZK satisfies perfect completeness if for any PPT adversary $\mathcal{A}$*

$$\Pr \left[\text{NIZK.Verify}(\text{vk}, x, \pi) = 1 \mid \begin{array}{l} \text{pp} \leftarrow \text{NIZK.Setup}(1^\lambda), \\ (s, (x, w)) \leftarrow \mathcal{A}(\text{pp}), \\ (\text{pp}, s, x, w) \in \mathcal{R}, \\ (\text{pk}, \text{vk}) \leftarrow \text{NIZK.KeyGen}(\text{pp}, s), \\ \pi \leftarrow \text{NIZK.Prove}(\text{pk}, x, w) \end{array} \right] = 1.$$

Definition 18 (Knowledge Soundness). *A NIZK satisfies knowledge soundness if for all PPT adversaries $\mathcal{A}$ there exists a PPT extractor $\mathcal{E}$ such that for any randomness ρ*

$$\Pr \left[\begin{array}{l} \text{NIZK.Verify}(\text{vk}, x, \pi) = 1, \\ (\varphi, x, w) \notin \mathcal{R}(\text{pp}) \end{array} \mid \begin{array}{l} \text{pp} \leftarrow \text{NIZK.Setup}(1^\lambda), \\ (\varphi, (x, w)) \leftarrow \mathcal{A}(\text{pp}; \rho), \\ (\text{pk}, \text{vk}) \leftarrow \text{NIZK.KeyGen}(\text{pp}, \varphi), \\ w \leftarrow \mathcal{E}(\text{pp}; \rho) \end{array} \right] = \text{negl}(\lambda).$$

Definition 19 (Succinctness). *An NIZK system is succinct (denoted as SNARK) if the size of the proof π is poly-logarithmic in the size of the witness w .*

Definition 20 (Zero-Knowledge). An NIZK satisfies zero-knowledge if there exists PPT simulator $\mathcal{S}$ such that for all PPT adversaries $\mathcal{A}$

$$\left\{ (\text{pp}, s, u, \pi) \left| \begin{array}{l} \text{pp} \leftarrow \text{NIZK.Setup}(1^\lambda), \\ (\varphi, (x, w)) \leftarrow \mathcal{A}(\text{pp}), \\ (\varphi, x, w) \in \mathcal{R}(\text{pp}), \\ (\text{pk}, \text{vk}) \leftarrow \text{NIZK.KeyGen}(\text{pp}, s), \\ \pi \leftarrow \text{NIZK.Prove}(\text{pk}, x, w) \end{array} \right. \right\} \approx \left\{ (\text{pp}, \varphi, x, \pi) \left| \begin{array}{l} (\text{pp}, \tau) \leftarrow \mathcal{S}(1^\lambda), \\ (\varphi, (x, w)) \leftarrow \mathcal{A}(\text{pp}), \\ (\varphi, x, w) \in \mathcal{R}(\text{pp}), \\ (\text{pk}, \text{vk}) \leftarrow \text{NIZK.KeyGen}(\text{pp}, s), \\ \pi \leftarrow \mathcal{S}(\text{pp}, x, \tau) \end{array} \right. \right\}.$$

E. Definitions for Malleable NIZK

We adopt the definitions of malleable NIZKs from [Chase et al.(2012)], [Chakraborty et al.(2025)] with the succinctness property included.

Definition 21 (Admissible Transformation). We define an n -ary transformation admissible for an NP relation $\mathcal{R}$ as $T = (T_x, T_w)$, where $\forall i \in [n] : (x_i, w_i) \in \mathcal{R} \Rightarrow (T_x(x_1, \dots, x_n), T_w(w_1, \dots, w_n)) \in \mathcal{R}$. Moreover, T_x and T_w are efficiently computable and for all statements $x_1, \dots, x_n$, we have $|T| + |x_1| + \dots + |x_n| \leq \text{poly}(|T_x(x_1, \dots, x_n)|)$ for a fixed polynomial poly .

We say that a set of transformations $\mathcal{T}$ is allowable for $\mathcal{R}$ if every $T \in \mathcal{T}$ is admissible for $\mathcal{R}$ and membership in $\mathcal{T}$ is efficiently decidable.

Definition 22 (Malleable Argument System). An argument system $\text{NIZK} = (\text{Setup}, \text{Prove}, \text{Verify})$ for $\mathcal{R}$ is malleable for an allowable set of transformations $\mathcal{T}$ if there is a PPT algorithm ZKEval that inputs the public parameters pp , an n -ary transformation $T = (T_x, T_w) \in \mathcal{T}$ and n statements with arguments $\{(x_i, \pi_i)\}_{i \in [n]}$, and outputs an argument π^* for $x^* = T_x(x_1, \dots, x_n)$.

A malleable argument system should retain completeness and controlled-malleable simulation extractability. Concretely, the completeness requires that given n instance-witness pairs, if each (x_i, w_i) for $i \in [n]$ is valid for $\mathcal{R}$, the proof (x^*, π^*) is also valid for $\mathcal{R}$ overwhelmingly.

For soundness, Chakraborty et al. [Chakraborty et al.(2025)] introduced a new definition as a relaxation of simulation extractability, where the extractor can provide a valid “explanation” of the statement instead of a witness. The formal definition is given in Definition 16 in Appendix C.

Next, we highlight the derivation privacy property, which is key to the unlinkability property for credential schemes. Derivation privacy guarantees that any PPT adversary can not distinguish the output of ZKEval from a freshly generated proof for the same statement. The definition of a malleable argument system with a privacy property.

Definition 23 (Derivation Privacy). A malleable NIZK system $\Pi = (\text{Setup}, \text{Prove}, \text{Verify})$ with associated simulator $S = (S_1, S_2)$ is derivation private if for every PPT adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ we have

$$\text{Adv}_{\mathcal{A}, \Pi}^{\text{rdp}}(\lambda) := |\Pr[\text{Exp}_{\mathcal{A}, \Pi}^{\text{rdp}} = 1] - \frac{1}{2}| \leq \text{negl}(\lambda)$$

for a negligible function negl where the experiment is defined in Figure 7 in Appendix C.

Definition 24 (Adversarial One-Way Function). An adversarial one-way function (AOWF) contains a generator AOWFGen inputs 1^λ and outputs public parameters pp describing sets $\mathcal{X}_{\text{pp}}$ and $\mathcal{R}_{\text{pp}}$ from which can be drawn efficiently uniformly samples and an efficiently computable function $f_{\text{pp}} : \mathcal{X}_{\text{pp}}^* \times \mathcal{R}_{\text{pp}} \rightarrow \mathcal{X}_{\text{pp}}$ (the AOWF). If there exists a polynomial p such that for every two-stage PPT adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$

$$\Pr[\text{valid}_{\text{pp}}(G, y) \wedge \neg \text{good}_{\text{pp}}(G, p(t))] \leq \text{negl}(\lambda)$$

for a negligible function negl where the probability is taken over $\text{pp} \leftarrow \text{AOWFGen}(1^\lambda)$, $(y, \text{st}) \xleftarrow{t} \mathcal{A}_1(1^\lambda, \text{pp})$, and $G \leftarrow \mathcal{A}_2(y, \text{st})$, t is the runtime of $\mathcal{A}_1(1^\lambda, \text{pp})$. Then we say AOWFGen satisfies adversarially one-wayness.

Theorem 2 (Recursive Construction for Malleable NIZK [Chakraborty et al.(2025)]). There exists a recursive construction for a malleable NIZK from a NIZK. The malleable NIZK is complete if the underlying NIZK is complete. The malleable NIZK is controlled-malleable simulation extractable if the underlying NIZK is simulation extractable, and AOWFGen is adversarial one-way and collision-resistant. The malleable NIZK is derivation private if the underlying NIZK is zero-knowledge and AOWFGen is unlinkable, which is also denoted as malleable NIZK.

F. Definitions for Plonk

We describe the Plonkish constraint system below. Let $\mathcal{C} : \mathbb{F}_p^{n+m} \rightarrow \mathbb{F}_p$ be an arithmetic circuit with a total of s gates, where each gate has fan-in two and can be either an addition or multiplication gate. Besides, Plonkish also supports circuits with a custom gate $G : \mathbb{F}_p^2 \rightarrow \mathbb{F}_p$ (also named as high-degree gates since each input can have degree of d), which is denoted as $\mathcal{C}[G]$. Let $\mathbf{x} \in \mathbb{F}_p^n$ be a public input to the circuit. In plonkish, we represent the resulting computation as a sequence of $n + s + 1$ triples:

$$(L, R, O) = \{(L_i, R_i, O_i) \in \mathbb{F}_p^3\}_{i=0, \dots, n+s}, \quad (1)$$

where L, R, O are $(n + s + 1)$ -size vector based on field $\mathbb{F}_p$. Concretely, the first n elements encode the public inputs; the next s rows represent the two values of fan-ins and one value of fan-out for each gate; and the final row enforces the equality between the final output and zero. To arithmetize the matrix, the prover in Plonk interpolates each cell in columns L, R, O into a univariate polynomial based on a cyclic subgroup (e.g., a smooth multiplicative group with order as the power of 2.) $H \subset \mathbb{F}_p$ where $H = \{w^1, \dots, w^{n+s+1}\}$ where $w^{n+s+1} = 1$, which derives three degree- $(n + s + 1)$ polynomial $f_L(X), f_R(X), f_O(X) \in \mathbb{F}_p[X]$ such that

$$f_L(w^i) = L_i, f_R(w^i) = R_i, f_O(w^i) = O_i, \text{ for } i \in [0, n + s]$$

Next, the prover needs to convince the verifier that the polynomial $f_L(X), f_R(X), f_O(X)$ encode a valid computation trace of the circuit $\mathcal{C}$, which can be ensured by (1) gate constraints that ensure each gate computes correctly, and (2) wiring constraints that ensures each wire connects correctly.

- Gate constraints: to prove that all gates were computed correctly, six selector polynomials $q_L(X), q_R(X), q_O(X), q_M(X), q_G(X), q_C(X) \in \mathbb{F}_p^{n+s+1} \rightarrow \{0, 1\}$ are introduced. The prover convinces the verifier that the following equation holds for all $x \in H$, where $\varphi(X)$ is the polynomial for public inputs, $q_G(X)$ is the selector polynomial for the custom gate.

$$q_L(X)f_L(X) + q_R(X)f_R(X) + q_M(X)f_L(X)f_R(X) + q_G(X)(G(f_L(X), f_R(X))) - q_O(X)f_O(X) + \varphi(X) = 0$$

- Wiring constraints: despite the computation correctness of each gate, the prover also needs to show that the wiring is correct, e.g., the output of one gate is connected to the left input of another. Concretely, the prover convinces the verifier that $\vec{f}_L || \vec{f}_R || \vec{f}_O = \vec{f}_L' || \vec{f}_R' || \vec{f}_O'$, where $\vec{f}_L', \vec{f}_R', \vec{f}_O'$ are vectors after permutation.

For the gate constraint, the prover can utilize a zero check, which allows the verifier to check the equality over H at a random queried point. The wiring constraint is trickier, the prover in Plonk first adds three index vectors for each column to indicate the position of the element, e.g.,

$$\vec{id}_L = (1, w, \dots, w^{n+s}), \vec{id}_R = k_1 \cdot \vec{id}_L, \vec{id}_O = k_2 \cdot \vec{id}_L$$

Correspondingly, the permuted index vectors are $\vec{\sigma}_L, \vec{\sigma}_R, \vec{\sigma}_O$. As a result, the prover can combine the tuple that consists of witness vector and index vector with random values β, γ as follows (in polynomials)

$$f(X) = \prod_{T \in \{L, R, O\}} (q_T(X) + \beta \cdot id_T(X) + \gamma), \quad (2)$$

$$g(X) = \prod_{T \in \{L, R, O\}} (q_T(X) + \beta \cdot \sigma_T(X) + \gamma), \quad (3)$$

and convince the verifier that $\prod_{x \in H} f(x) = \prod_{x \in H} g(x)$ with a grand product check.

In the original paper of Plonk [Gabizon et al.(2019)], the authors presented the SNARK construction directly without an IOP. In this paper, we focus on the IOP construction of Plonk for a better discussion on the following recursive SNARK.

Construction 4 (Plonk Interactive Oracle Proof Construction). *We describe the Plonk's IOP protocol as follows, where $[\cdot]$ denotes a polynomial oracle and RO denotes the random oracle.*

- Preprocessing: The prover and verifier construct the following polynomial oracles

$$[q_L(X)], [q_R(X)], [q_O(X)], [q_M(X)], [q_G(X)], [q_C(X)], \\ [\sigma_L(X)], [\sigma_R(X)], [\sigma_O(X)].$$

- The prover constructs $f_L(X), f_R(X), f_O(X), \varphi(X)$ satisfying the following equation and sends $[f_L(X)], [f_R(X)], [f_O(X)],$

$$q_L(X)f_L(X) + q_R(X)f_R(X) + q_M(X)f_L(X)f_R(X) + q_G(X)(G(f_L(X), f_R(X))) - q_O(X)f_O(X) + \varphi(X) = 0$$

- The verifier queries RO and sends back β, γ .
- The prover constructs $z(X)$ satisfying the following equation and sends $[z(X)]$,

$$L_0(X)(z(X) - 1) = 0, \quad z(w \cdot X)g(X) - z(X)f(X) = 0$$

- The verifier queries RO and sends back α .
- The prover computes $h(X)$, its quotient polynomial is $t(X) = h(X)/z_H(X)$, and sends $[t(X)]$, where $h(X) :=$

$$\begin{aligned} & q_L(X)f_L(X) + q_R(X)f_R(X) + q_M(X)f_L(X)f_R(X) \\ & + q_G(X)(G(f_L(X), f_R(X))) - q_O(X)f_O(X) + \varphi(X) \\ & + \alpha(z(wX)g(X) - z(X)f(X)) + \alpha^2(L_0(X)(z(X) - 1)), \end{aligned}$$

where $f(X), g(X)$ are computed as per Equation 2.

- The verifier queries RO and sends back ζ .
- The verifier queries all the oracles above and obtains

$$\begin{aligned} & - f_L = f_L(\zeta), f_R = f_R(\zeta), f_O = f_O(\zeta) \\ & - q_L = q_L(\zeta), q_R = q_R(\zeta), q_O = q_O(\zeta), \\ & \quad q_M = q_M(\zeta), q_G = q_G(\zeta), q_C = q_C(\zeta) \\ & - z_1 = z(w \cdot \zeta), z_2 = z(\zeta), t = t(\zeta) \end{aligned}$$

- Besides, the verifier computes the following locally

$$\begin{aligned} & - f = (f_L + \beta \cdot \zeta + \gamma)(f_R + \beta \cdot k_1 \cdot \zeta + \gamma)(f_O + \beta \cdot k_2 \cdot \zeta + \gamma) \\ & - g = (f_L + \beta \cdot \sigma_L + \gamma)(f_R + \beta \cdot \sigma_R + \gamma)(f_O + \beta \cdot \sigma_O + \gamma) \\ & - z_1 = z(w \cdot \zeta), z_2 = z(\zeta) \\ & - L_0(\zeta), z_H(\zeta), \varphi(\zeta) \end{aligned}$$

- Finally, the verifier checks whether $t \cdot z_H(\zeta)$ equals to

$$\begin{aligned} & q_L \cdot f_L + q_R \cdot f_R + q_M \cdot f_L \cdot f_R + q_G \cdot G(f_L, f_R) - q_O \cdot w_O \\ & + q_C + \varphi(\zeta) + \alpha \cdot (z_1 g - z_2 g) + \alpha^2 \cdot (L_0(\zeta)(z_2 - 1)) \end{aligned}$$

Example 1 (Malleable PoP for BLS Signatures). The BLS signature scheme Sig contains a tuple of algorithms:

- $\text{pp} \leftarrow \text{Sig.Setup}(1^\lambda)$.
- $(sk, vk) \leftarrow \text{Sig.KeyGen}(\text{pp})$: the key generation algorithm outputs signing key $sk \leftarrow \mathbb{F}_p$ and verification key $vk := g_2^{sk} \in \mathbb{G}_2$.
- $\sigma \leftarrow \text{Sig.Sign}(sk, m)$: the signing algorithm outputs the signature $\sigma := c^{sk} \in \mathbb{G}_1$ where $c = H(m)$ and H is a hash function mapping m into $\mathbb{G}_1$.
- $b \leftarrow \text{Sig.Verify}(vk, m, \sigma)$: the verification algorithm runs the two steps:
 - $V_1(vk, \sigma, m) \rightarrow (c, \perp)$: it outputs $c := H(m)$;
 - $V_2(vk, \sigma, c, \perp) \rightarrow b$: it outputs a bit $b = 1$ if the equation $e(c, vk) = e(\sigma, g_2)$ holds or $b = 0$ otherwise.

We define the corresponding transformation T and relaxed verification algorithm V'_2 as follows:

- $T_\Sigma(\sigma, r_s) \rightarrow \sigma'$: outputs $\sigma' := \sigma + R_s$ where $r_s = (R_s, r_1)$, $R_s = g_1^{r_1}$, and g_1 is a public generator of $\mathbb{G}_1$;
- $T_{rand}(c, \perp, r_c) \rightarrow c'$: outputs $c' := c + R_c$ and $r_c = (R_c, r_2)$ where $R_c = g_1^{r_2}$;
- V'_2 : outputs a bit $b = 1$ if the equation $e(c', vk) \cdot e(R_s, g_2) = e(\sigma', g_2) \cdot e(R_c, vk) \wedge R_s = g_1^{r_1} \wedge R_c = g_1^{r_2} \wedge \sigma' = \sigma + R_s$ holds or $b = 0$ otherwise.

Example 2 (Rerandomizable PoP scheme for ECDSA Signature). The ECDSA signature Sig contains a tuple of algorithms:

- $\text{pp} \leftarrow \text{Sig.Setup}(1^\lambda)$.
- $(sk, vk) \leftarrow \text{Sig.KeyGen}(\text{pp})$: the key generation algorithm outputs signing key $sk \leftarrow \mathbb{F}_p$ and verification key $vk := g^{sk} \in \mathbb{G}$.
- $\sigma \leftarrow \text{Sig.Sign}(sk, m)$: the signing algorithm runs the following steps:
 - samples $k \leftarrow \mathbb{F}_p$ and computes $K = g^k = (r, \cdot)$;
 - computes $s = (c + r \cdot sk)/k$;
 - outputs signature $\sigma = (r, s)$.
- $b \leftarrow \text{Sig.Verify}(vk, m, \sigma)$: the verification algorithm runs the two steps:
 - $V_1(vk, \sigma, m) \rightarrow (c, (u_1, u_2, K))$: It parses σ into (r, s) , recovers K via r , and outputs $c := H(m)$ and $(u_1 := c/s - c, u_2 = r/s, K)$;
 - $V_2(vk, \sigma, c, (u_1, u_2, K)) \rightarrow b$: It outputs a bit $b = 1$ if $g^{u_1+c} + vk^{u_2} = K \wedge K.x = \sigma.r$ or $b = 0$ otherwise.

We define the corresponding transformation T and relaxed verification algorithm V'_2 as follows:

- $T_\Sigma(\sigma, r_s) \rightarrow \sigma'$: it parses σ into (r, s) and r_s into (r_0, H) where $H := h^{r_0}$ and $h \in \mathbb{G}, r_0 \in \mathbb{F}_p$. It outputs $\sigma' := (K'.x, \perp)$ where $K' := K + H$;
- $T_{rand}(c, \text{aux}, r_c) \rightarrow (c', \text{aux}')$: It parses r_c into (r_1, r_2, r_3, H) and aux into (u_1, u_2, K) , calculates $K' = H + K$, outputs $c' = c + r_1, u'_1 = u_1 + r_2, u'_2 = u_2 + r_3$, and outputs $(c', (u'_1, u'_2, K'))$;
- V'_2 : it outputs a bit $b = 1$ if the equation $g^{c+u_1} + vk^{u_2} + h^{r_0} = K + g^{r_1+r_2} + vk^{r_3} \wedge H = h^{r_0} \wedge K = K_0 + H \wedge K_0.x = \sigma.r$ or $b = 0$ otherwise.

Example 3 (Rerandomizable PoP scheme for RSA Signature). The RSA signature scheme Sig contains a tuple of algorithms:

- $\text{pp} \leftarrow \text{Sig.Setup}(1^\lambda)$.
- $(sk, vk) \leftarrow \text{Sig.KeyGen}(\text{pp})$: the key generation algorithm outputs signing key $(sk, vk = (pk, n))$ such that $sk \cdot pk = 1 \pmod{(p-1)(q-1)}$, $n = p \cdot q$ where p, q are big primes.
- $\sigma \leftarrow \text{Sig.Sign}(sk, m)$: the signing algorithm runs:
 - computes $c = H(m)$;
 - outputs the signature $\sigma = c^{sk} \pmod{n}$.
- $b \leftarrow \text{Sig.Verify}(vk, m, \sigma)$: the verification algorithm runs the two steps:
 - $V_1(vk, \sigma, m) \rightarrow (c, \perp)$: It outputs $c = \text{Sig.Verify}(vk, m, \sigma)$;
 - $V_2(vk, \sigma, c, \perp) \rightarrow b$: It directly outputs the bit c from V_1 ;

We define the corresponding transformation T and relaxed verification algorithm V'_2 as follows:

- $\perp \leftarrow T_\Sigma(\sigma, \perp)$: it directly outputs $\perp$;
- $(c, \perp) \leftarrow T_{rand}(c, \perp, \perp)$: it directly outputs $(c, \perp)$;
- V'_2 : it outputs a bit $b = 1$ if $c = 1$.

Remark 4. Strictly speaking, the malleable PoP for RSA signatures degrades to the trivial one as Construction 2. This is because the RSA signature scheme does not support any meaningful rerandomization operation. However, we include this instantiation to demonstrate the versatility of our construction.

Example 4 (Rerandomizable PoP scheme for Schnorr Signature). The Schnorr signature Sig contains a tuple of algorithms:

- $\text{pp} \leftarrow \text{Sig.Setup}(1^\lambda)$.
- $(sk, vk) \leftarrow \text{Sig.KeyGen}(\text{pp})$: the key generation algorithm outputs signing key $sk \leftarrow \mathbb{F}_p$ and verification key $vk := g^{sk} \in \mathbb{G}$.
- $\sigma \leftarrow \text{Sig.Sign}(sk, m)$: the signing algorithm runs:
 - samples a random $r \leftarrow \mathbb{F}_p$ and computes $R = g^r$
 - computes $c = H(R, m)$ and $z = sk \cdot c + r$
 - outputs the signature $\sigma = (R, c, z)$
- $b \leftarrow \text{Sig.Verify}(vk, m, \sigma)$: the verification algorithm runs the two steps:
 - $V_1(vk, \sigma, m) \rightarrow (c, R)$: It parses σ into (R, c, z) , computes $c = H(R, m)$, and outputs c, R ;
 - $V_2(vk, \sigma, c, R) \rightarrow b$: It parses σ into $(\cdot, \cdot, z)$ and outputs a bit $b = 1$ if $g^z = R + vk^c$ or 0 otherwise;

We define the corresponding transformation T and relaxed verification algorithm V'_2 as follows:

- $(\cdot, \cdot, z') \leftarrow T_\Sigma((\cdot, \cdot, z), r_s)$: it parses r_s as $r_1 \in \mathbb{F}_p$ and outputs $z' = z + r_1$;
- $(c', \text{aux}') \leftarrow T_{rand}(c, \text{aux}, r_c)$: it parses aux as R and r_c as (H, r_2, r_3) , calculates $R' = R + H, c' = c + r_3$ where $h \in \mathbb{G}$ and $H = h^{r_2}$, and outputs (c', R') ;
- V'_2 : it outputs a bit $b = 1$ if the equation $g^z + h^{r_2} + vk^{r_3} = vk^c + R + g^{r_1} \wedge z = \sigma.z + r_1$.

To prove that the protocol Π_{Re2creds} realizes $\mathcal{F}_{\text{Re2creds}}$, we need to ensure there exists an ideal-world simulator $\mathcal{S}$ such that for any real-world adversary $\mathcal{A}$ and environment $\mathcal{E}$, the executions of $\mathcal{F}_{\text{Re2creds}}$ and Π_{Re2creds} are indistinguishable. We examine the scenarios where (1) all parties are uncorrupted, (2) the issuer $\mathcal{I}$ is corrupted, (3) the holder $\mathcal{H}$ is corrupted, (4) the verifier $\mathcal{V}$ is corrupted, and (5) all parties are corrupted.

G. All uncorrupted

In this case, when the uncorrupted issuer $\mathcal{I}$ is activated by a (SETUP, sid) message, $\mathcal{S}$ conducts as follows:

- Parse sid into $(\mathcal{I}, \text{sid}')$ and gives “ $\mathcal{I}$ ” input (Setup, sid).
- When “ $\mathcal{I}$ ” outputs (SETUPDONE, sid), $\mathcal{S}$ executes the following steps:
 - runs $(sk, vk) \leftarrow \Sigma.\text{KeyGen}(1^\lambda)$;
 - defines attribute spaces $\vec{\mathbb{A}}$;
 - define $\text{Present}(\text{ctx}, \mathcal{T}_\phi)$ as follows: sample $\vec{a} \in \vec{\mathbb{A}}$ such that $\phi(\vec{a}) = 1$ for any $\phi \in \mathcal{T}_\phi$ and calculate $\sigma \leftarrow \Sigma.\text{Sign}(sk, \vec{a})$.
Next, compute the presentation token pt as in the real-world protocol and return pt.
 - define $\text{Verify}(\text{pt}, \mathcal{T}_\phi, \text{ctx})$ as the real world algorithm that verifies with respect to vk .

- $\mathcal{S}$ sends (SETUP, sid, Present, Verify, $\vec{\mathbb{A}}$) to $\mathcal{F}$.

In $\mathcal{F}_{\text{Re2creds}}$, it checks the structure of sid and aborts if it is not equal to $(\mathcal{I}, \text{sid}')$. However, there is also the same operation in Π_{Re2creds} , which does not change the view of $\mathcal{E}$ with the simulator defined above.

When the uncorrupted issuer $\mathcal{I}$ is activated by a **ISSUE** message, $\mathcal{S}$ conducts as follows:

- It receives (ALLOWISSUE, sid, $\mathcal{H}_i$) from $\mathcal{F}_{\text{Re2creds}}$.
- $\mathcal{S}$ samples dummy attribute values $\vec{a} \in \vec{\mathbb{A}}$ and gives “ $\mathcal{I}$ ” input (ISSUE, sid, $\vec{a}$, $\mathcal{H}_i$).
- When “ $\mathcal{H}_i$ ” outputs (ISSUE, sid, $\vec{a}$), output (ALLOWISSUE, sid) to $\mathcal{F}$.

The simulator $\mathcal{S}$ lacks information to simulate the **ISSUE** interface when both parties are honest, but this difference is not noticeable because of the adoption of the secure communication channel.

When the uncorrupted holder $\mathcal{H}_i$ is activated by a **PRESENT** message, there is no network communication that needs to be simulated by $\mathcal{S}$ because credential presentation is non-interactive. Similarly, when the uncorrupted verifier $\mathcal{V}_i$ is activated by a **Verify** message, $\mathcal{S}$ simulates nothing. Now, we need to prove that the distributions of generated presentation tokens from two worlds are indistinguishable. Both the real-world protocol and **Present** generate a presentation by providing a Π and a signature. In the protocol Π_{Re2creds} , a credential holder generates a presentation by reusing the obtained credential against the specified ctx and predicates, while **Present** generates a new valid credential for each presentation. We know that the adopted proof systems ensure *zero-knowledge*, and malleable NIZK also provides *derivation privacy*. Therefore, we can conclude that this difference is not noticeable for $\mathcal{E}$. Besides, according to the definition of **Present**, we know that it is the same as the verification process in Π_{Re2creds} . Besides the presentation validity check, $\mathcal{F}$ also contains an unforgeability check for presentations. Now, we need to prove that the introduction of an unforgeability check does not affect the view of $\mathcal{E}$.

If the unforgeability check for presentations triggers with non-negligible probability, there must be a presentation token pt that was valid before but is rejected by the unforgeability check of $\mathcal{F}_{\text{Re2creds}}$. It denotes that one of the two conditions must hold with non-negligible probability:

- pt proves a valid credential that belongs to an honest credential holder with the correct attribute values, but it never creates a presentation under a specified context ctx.
- pt proves a knowledge of a credential that does not belong to an honest holder.

The above conditions can be reduced to signature forgery and NIZK or Sigma protocol proof forgery with a non-negligible probability. Consider the fact that the credential issuance and presentation generation processes occur only polynomial times, we can construct an attack instance for the unforgeability game for signature and proof forgery if the unforgeability check occurs in $\mathcal{F}_{\text{Re2creds}}$, which contradicts the assumption. Therefore, we can conclude that the output distributions of **Verify** in $\mathcal{F}_{\text{Re2creds}}$ with $\mathcal{S}$ and Π_{Re2creds} with $\mathcal{A}$ are indistinguishable.

H. Someone Corrupted

If the issuer $\mathcal{I}$ is corrupted, $\mathcal{S}$ should generate the transcripts of all the messages to the adversary $\mathcal{A}$ throughout the Π_{Re2creds} execution, along with all outputs transmitted to $\mathcal{F}_{\text{Re2creds}}$ from the corrupted $\mathcal{I}$. $\mathcal{S}$ runs as follows:

SETUP

- $\mathcal{S}$ notices the setup as it notices $\mathcal{I}$ registering a public key with “ $\mathcal{F}_{ca}$ ” with sid = $(\mathcal{I}, \text{sid}')$.
 - If the registered key is of the form (vk, π_{sk}) and π_{sk} is valid, $\mathcal{S}$ extracts sk from π_{sk} .
 - $\mathcal{S}$ defines **Present**, **Verify**, and $\vec{\mathbb{A}}$ as when $\mathcal{I}$ is honest with respect to the extracted sk .
 - $\mathcal{S}$ sends (Setup, sid) to $\mathcal{F}$ on behalf of $\mathcal{I}$.
- On input (Setup, sid) from $\mathcal{F}$.
 - $\mathcal{S}$ sends (SETUP, sid, Present, Verify, $\vec{\mathbb{A}}$) to $\mathcal{F}$.
- On input (SETUPDONE, sid) from $\mathcal{F}$.
 - $\mathcal{S}$ continues simulating “ $\mathcal{I}$ ”.

ISSUE

- $\mathcal{S}$ notices this issuance as it receives (ALLOWISSUE, sid, $\mathcal{H}_i$) from $\mathcal{F}$.
 - output (ALLOWISSUE, sid) to $\mathcal{F}$.
- $\mathcal{S}$ obtains (ISSUE, sid, $\vec{a}$, $\mathcal{H}_i$) as $\mathcal{I}$ is corrupt.
 - $\mathcal{S}$ gives “ $\mathcal{I}$ ” input (ISSUE, sid, $\vec{a}$, $\mathcal{H}_i$).

If the holder $\mathcal{H}_i$ is corrupted, $\mathcal{S}$ should generate the transcripts of all the messages to the adversary $\mathcal{A}$ throughout the Π_{Re2creds} execution, along with all outputs transmitted to $\mathcal{F}_{\text{Re2creds}}$ from the corrupted $\mathcal{H}_i$. $\mathcal{S}$ runs as follows:

ISSUE

- $\mathcal{S}$ notices this issuance as it receives (ALLOWISSUE, sid, $\mathcal{H}_i$) from $\mathcal{F}$.
 - output (ALLOWISSUE, sid) to $\mathcal{F}$.

- $\mathcal{S}$ obtains $\vec{a}$ as $\mathcal{H}_i$ is corrupt.
 - $\mathcal{S}$ gives “ $\mathcal{I}$ ” input (ISSUE, sid, $\vec{a}$, $\mathcal{H}_i$).

PRESENT

- $\mathcal{I}$ simulates nothing.

If the verifier is corrupted, $\mathcal{S}$ should generate the transcripts of all the messages to the adversary $\mathcal{A}$ throughout the Π_{Re2creds} execution, along with all outputs transmitted to $\mathcal{F}_{\text{Re2creds}}$ from the corrupted $\mathcal{V}_i$. Because the $\mathcal{F}_{\text{Re2creds}}$ mirrors the verification procedure of Π_{Re2creds} , $\mathcal{S}$ simulates nothing.

If all parties are corrupted, the simulator $\mathcal{S}$ should generate the transcripts of all messages to the adversary $\mathcal{A}$ throughout Π_{Re2creds} execution, along with all outputs transmitted to $\mathcal{F}_{\text{Re2creds}}$ from the corrupted parties. The simulation of $\mathcal{S}$ is a mixture of the above three scenarios.